

StoreNet 住宅虛擬電廠研究

公開資料驗證 論文復現與控制策略改進

研究報告 2026 年 9 月

這份研究從兩篇 StoreNet 論文出發。第一篇公開了愛爾蘭 20 戶住宅的用電、太陽能與電池資料；第二篇用混合整數線性規劃（MILP）安排電池的充放電，比較「省電費」和「降低尖峰」這些不同目標的結果。我們先確認公開資料能不能對上論文的圖表，再用這批資料把第二篇的方法做一遍，最後處理電池集中充電造成的新尖峰。

最主要的發現是：在這組日夜電價和模型設定下，電費同樣最低的排程，尖峰可以差很多。先把最低電費固定住，再要求尖峰越低越好，代表日的社區購電尖峰可以從 75.50 kW 降到 19.35 kW；45 個月度取樣日的逐日降幅平均 65.3%，電費在計算誤差範圍內完全相同。如果還想把尖峰壓得更低，就要開始多付電費。

後面的內容照研究的順序來寫。先看官方公開的資源和資料論文的 Figure 5 到 Figure 10，再說明 MILP 的設定、五種策略，以及方法論文 Figure 5、Figure 6、Figure 7 和 Table I 的對照。接著從作者的結論引出 Peak Guard，說明用澳洲資料檢查時看到的線索，以及第二版模型怎麼把省電費和削峰的關係算清楚。

文中的「原圖」是指論文裡刊出的圖；「我們的結果」是指用公開資料和我們自己的程式算出來的結果。兩篇論文都有 Figure 5 這類相同的圖號，所以每次引用都會先寫明是資料論文還是方法論文。

資料論文：Trivedi 等人 2024 [1] 方法論文：Bahloul 等人 2022 [2]
報告中的數字整理自 2026 年 9 月 4 日為止的結果檔。

內容導覽

第一部分 資源與資料論文	3
公開資源與下載的檔案	3
資料檔案實際長什麼樣	4
資料論文 Figure 5 資料是否齊全	5
資料論文 Figure 6 功率與能量是否一致	6
H4 的資料問題與我們的處理	7
資料論文 Figure 7 年用電與氣溫	8
Figure 7b 的單位為何需要修正	9
資料論文 Figure 8 有太陽能住戶的能量流	10
資料論文 Figure 9 H4 一週的電池運作	11
資料論文 Figure 10 電壓不平衡	12
第二部分 方法論文與獨立復現	13
MILP 的問題與基本設定	13
電力流向與電池電量怎麼計算	14
五種策略各自在做什麼	15
代表日與省錢率的比較基準	16
方法論文 Figure 5 代表日原圖	17
方法論文 Figure 6 月度原圖	20
方法論文 Figure 7 容量與功率敏感度	22
方法論文 Table I 完整對照	23
作者的總結與留下的問題	24
第三部分 控制策略改進	25
Peak Guard 加上一條尖峰上限	25
澳洲資料怎麼轉成同一個控制問題	27
澳洲結果為何讓我們重新思考	28
第二版模型先把免費的削峰找出來	29
再降低尖峰要付多少電費	31
需量電價會讓聚合商選到哪裡	33
用公開電池資料檢查效率假設	34
總結與接下來的研究	35
附錄 策略算式與 MATLAB 程式	36
參考資料與結果來源	51

第一部分 資源與資料論文

公開資源與下載的檔案

StoreNet 的實驗場在愛爾蘭的 Dingle 半島。20 戶住宅各有一顆 10 kWh、3.3 kW 的電池，其中 10 戶有裝太陽能。Trivedi 等人的資料論文在 2024 年刊出，公開的是 2020 年整年的一分鐘資料；論文和下載頁面分別列在參考資料 [1] 和 [3]。

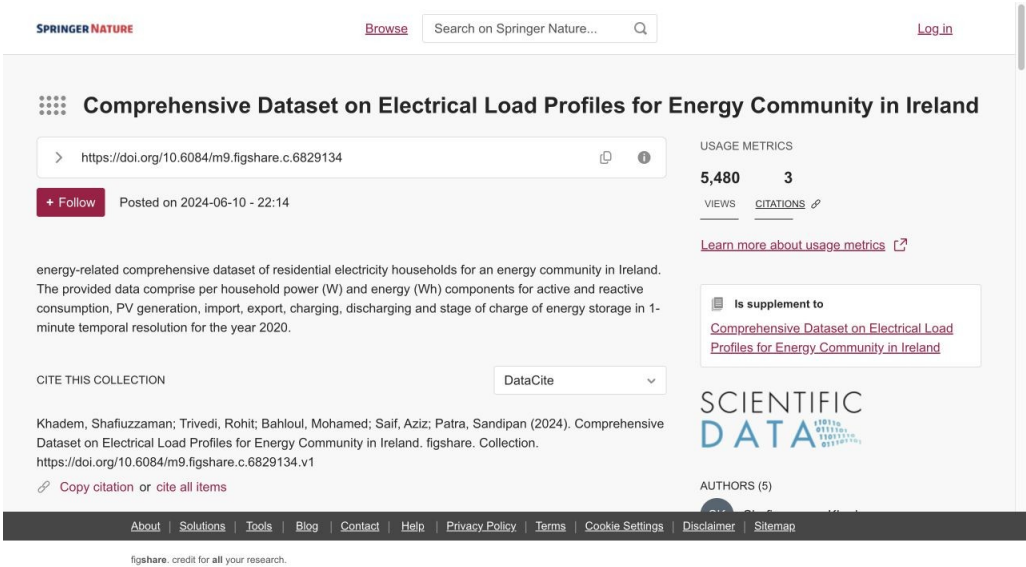


圖 1 官方 Figshare 資料集合頁面。集合名稱與 DOI 可以用來確認下載來源。

來源：Springer Nature Figshare，2026 年 9 月 5 日擷取 [3]。

表 1 下載資源的組成

資源	數量	用途
住戶功率檔 H1_W 至 H20_W	20 個 CSV	查看功率、SoC 與資料狀態
住戶能量檔 H1_Wh 至 H20_Wh	20 個 CSV	重算用電、PV、電池及電網交換
氣象資料	1 個 CSV	氣溫及其他氣象欄位
作者說明與程式	1 份 README 及 4 支 Python	理解處理流程與繪圖方式

我們核對過 46 個公開檔案的大小、官方提供的 MD5，以及自己算的 SHA-256。這證明手上的檔案和官方發布的版本一模一樣；但檔案裡面仍可能有插值或時間對不齊的問題，後面會一張圖一張圖檢查。

資料檔案實際長什麼樣

官方下載檔案的本地表格檢視

H1 前五筆紀錄的部分欄位 原始欄名與數值均保留

H1_W.csv

date	Production (W)	Consumption (W)	Charge (W)	H1_W
2020-01-01 01:00:00	0.0	95.0	8.0	1.0
2020-01-01 01:01:00	0.0	95.0	8.0	1.0
2020-01-01 01:02:00	0.0	95.0	8.0	1.0
2020-01-01 01:03:00	0.0	94.0	8.0	1.0
2020-01-01 01:04:00	0.0	93.0	8.0	1.0

H1_Wh.csv

date	Production (Wh)	Consumption (Wh)	From grid (Wh)	Feed-in (Wh)
2020-01-01 01:01:00	0.0	1.58	1.71	0.0
2020-01-01 01:02:00	0.0	1.58	1.71	0.0
2020-01-01 01:03:00	0.0	1.57	1.7	0.0
2020-01-01 01:04:00	0.0	1.55	1.68	0.0
2020-01-01 01:05:00	0.0	1.55	1.68	0.0

此處僅展示部分欄位；完整 CSV 另含放電、電池電量等資訊。

圖 2 下載後的 CSV 內容截圖。為了看清楚欄位，這裡只顯示 H1 的部分欄位和前五筆紀錄。

來源：官方檔案 H1_W.csv、H1_Wh.csv 直接打開的內容；數值沒有改過。

W 是功率，代表某一個時間點用電或發電的快慢；Wh 是一段時間累積下來的能量。這批 Wh 檔每一列對應一分鐘，所以要把那一列的 Wh 乘以 60，才是這一分鐘的平均功率（W）；想得到一天的 kWh，就把當天的 Wh 加總再除以 1,000。

後面的最佳化真正用到的輸入只有兩欄：Consumption（住戶用電）和 Production（太陽能可以發的電）。Charge、Discharge、From grid、Feed-in 和 State of Charge 記錄的是現場控制器實際的運作結果，我們拿它們來檢查資料，或是當作 SB-SC 這個現場控制器的實測基準。

作者提供了 data_processing.py、power plots.py、energy plots.py 和 setup.py 四支程式。這些程式讓我們看得出每張圖是怎麼算的，但裡面寫死了作者電腦的路徑，也引用了一些沒有公開的原始檔。所以我們保留作者的程式不動，另外寫一套自己能跑的重現流程，並且把「照原程式算」和「修正單位後算」的結果分開存。

欄位說明與作者程式：[3]；我們的繪圖程式：src/reproduce_data_paper_figures.py。

資料論文 Figure 5 資料是否齊全

第一件要確認的事，是每一戶的資料有沒有一直記錄下來。圖中每一橫列是一戶，藍色代表那一分鐘的功率檔有可用的資料，白色代表沒有。左邊的百分比是整年共同時間軸上有資料的比例。

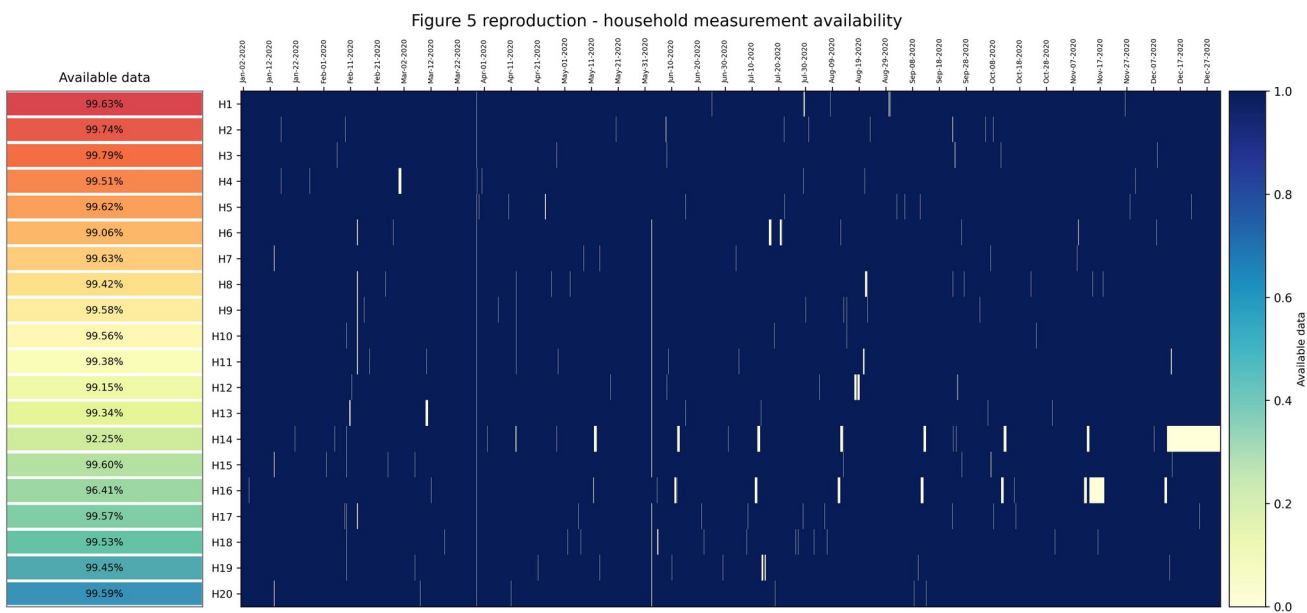


圖 3 我們重畫的資料論文 Figure 5。H14 與 H16 的缺口較明顯。

來源：官方 W 檔與作者判斷資料狀態的方式；資料論文 [1] Figure 5，結果檔見 [5]。

20 戶的百分比都能重現到論文寫的小數點後兩位。H14 是 92.25%，H16 是 96.41%，其他住戶都超過 99%。H14 的資料到 12 月 11 日就結束了，所以最後那段白色不能當成「年底用電是零」。

作者在文字裡只提到幾個短暫的缺口，但實際情況不只這樣。程式檢查還找到 H4 和 H13 有整天沒資料的日子，H16 也有更長的連續缺口。圖縮小以後，缺口看起來可能只是一條細線，所以挑模擬日期時要回到逐分鐘的紀錄去檢查，不能只看圖。

表 2 Figure 5 的核對重點

檢查項目	結果
論文可用率	20 戶全部對到兩位小數
共同時間軸的長度	527,040 列
主要用途	篩除不完整日期，避免把缺資料當成低負載

資料論文 Figure 6 功率與能量是否一致

Figure 6 把同一時間的 W 和 $Wh \times 60$ 分別放在橫軸和縱軸。如果兩者一致，點就會貼著對角線。原圖用 H4 當例子，點幾乎排成一條線；我們拿公開的 H4 檔案做同樣的比較，點卻明顯散開。

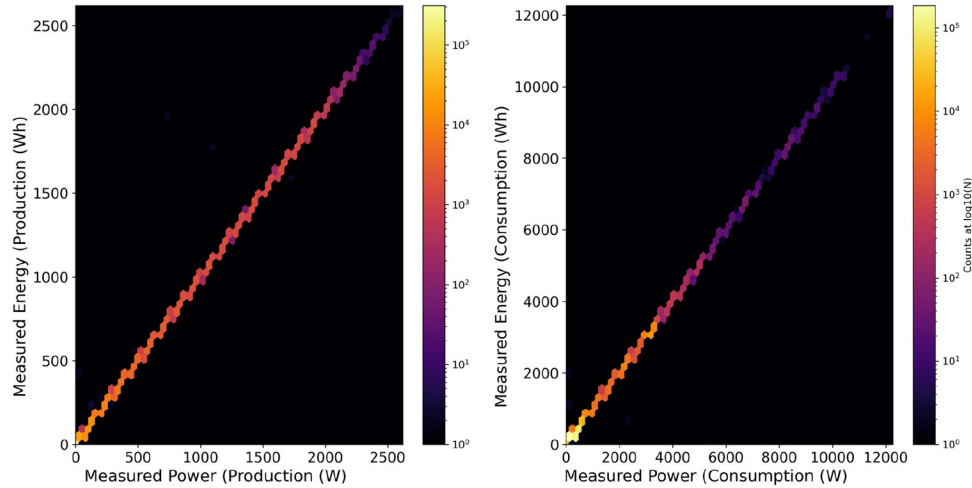


圖 4 資料論文 Figure 6 原圖。顏色表示同一區域的資料點數。

來源：Trivedi 等人 [1]，第 7 頁 Figure 6。

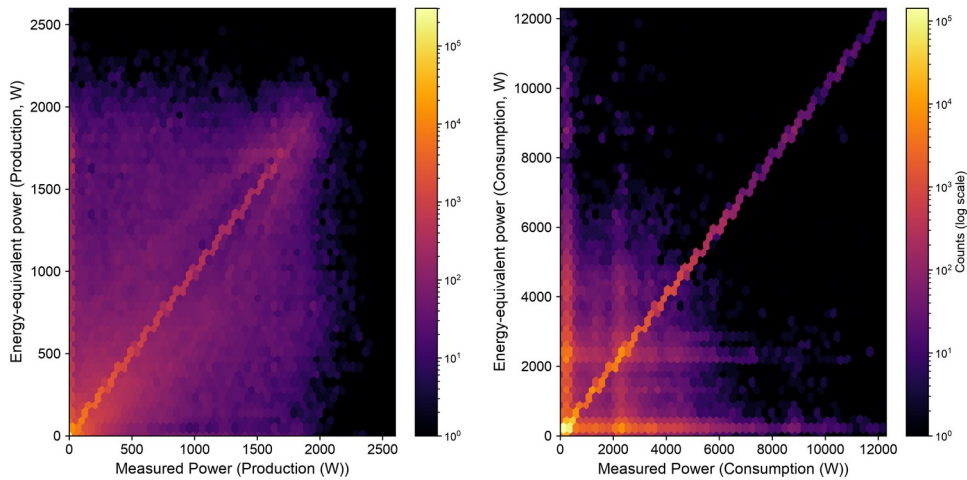


圖 5 用公開 H4 資料畫出的結果。座標範圍和上方原圖一樣，但點沒有原圖那麼集中。

來源：H4_W.csv 與 H4_Wh.csv；結果檔見 [5]。超出圖框的點仍然有算進相關係數。

這個差異我們照實保留。公開 H4 的用電相關係數約 0.522，PV 約 0.732；而作者畫 Figure 6 的程式讀的是一個沒有公開的原始檔（90962 W / Wh）。所以我們能用公開資料重畫這張檢查圖，但沒辦法說已經把刊出的 H4 原圖一點一點對上。

H4 的資料問題與我們的處理

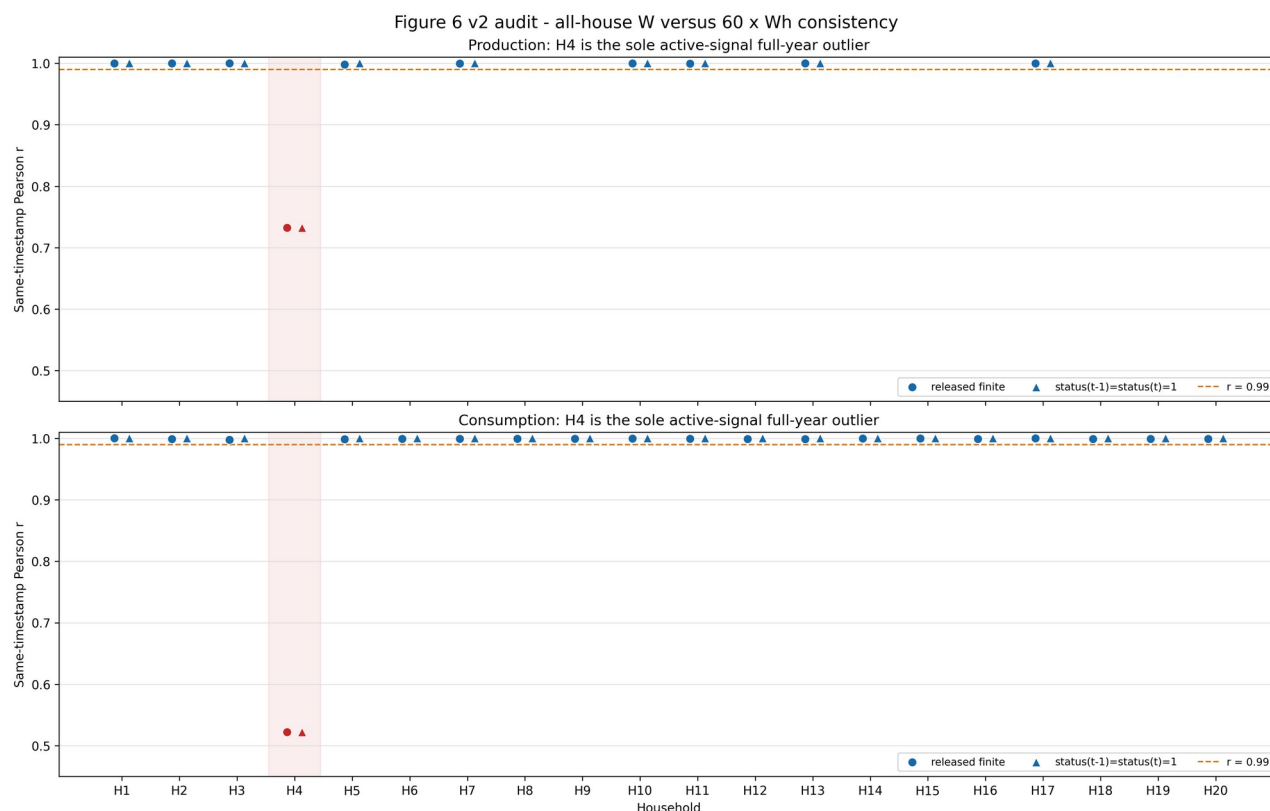


圖 6 全部住戶的 W 與 Wh×60 一致性。多數住戶的相關係數接近 1，H4 是明顯例外。

來源：結果檔見 [5]；用電 20 戶、PV 10 戶。

把同樣的檢查套到其他住戶，用電有 19 / 20 戶、PV 有 9 / 10 戶的相關係數超過 0.99，例外全部集中在 H4。H4_W 還多了一個沒有說明的 Unnamed: 6 欄位。逐日檢查後發現時間大約錯開一小時，另外有 42 個區塊的日期看起來是「日」和「月」對調了；還有 35 天用這兩條規則都解釋不了。

這些線索足以說明公開檔有時間對不齊的問題，但不夠讓我們還原作者原本的處理過程。我們沒有自作主張把整段資料平移或把日期換回來。主要分析照用公開的 Wh 檔，另外在代表日多做一組把 H4 拿掉的 19 戶 / 9 PV 敏感度分析。Wh 檔的能量收支比較一致，但這也不代表它就是正確的原始資料。

表 3 公開檔同時間比較

訊號	H4 相關係數	其他住戶
用電	0.522	19 戶皆高於 0.99
PV	0.732	9 戶皆高於 0.99

資料論文 Figure 7 年用電與氣溫

Figure 7 分成兩部分。上圖比較各戶在整段公開資料裡的用電量；下圖把整個社區的用電和氣溫放在同一條時間軸上。兩張圖都能照作者的程式重畫，但下圖的用電單位有問題，要另外處理。

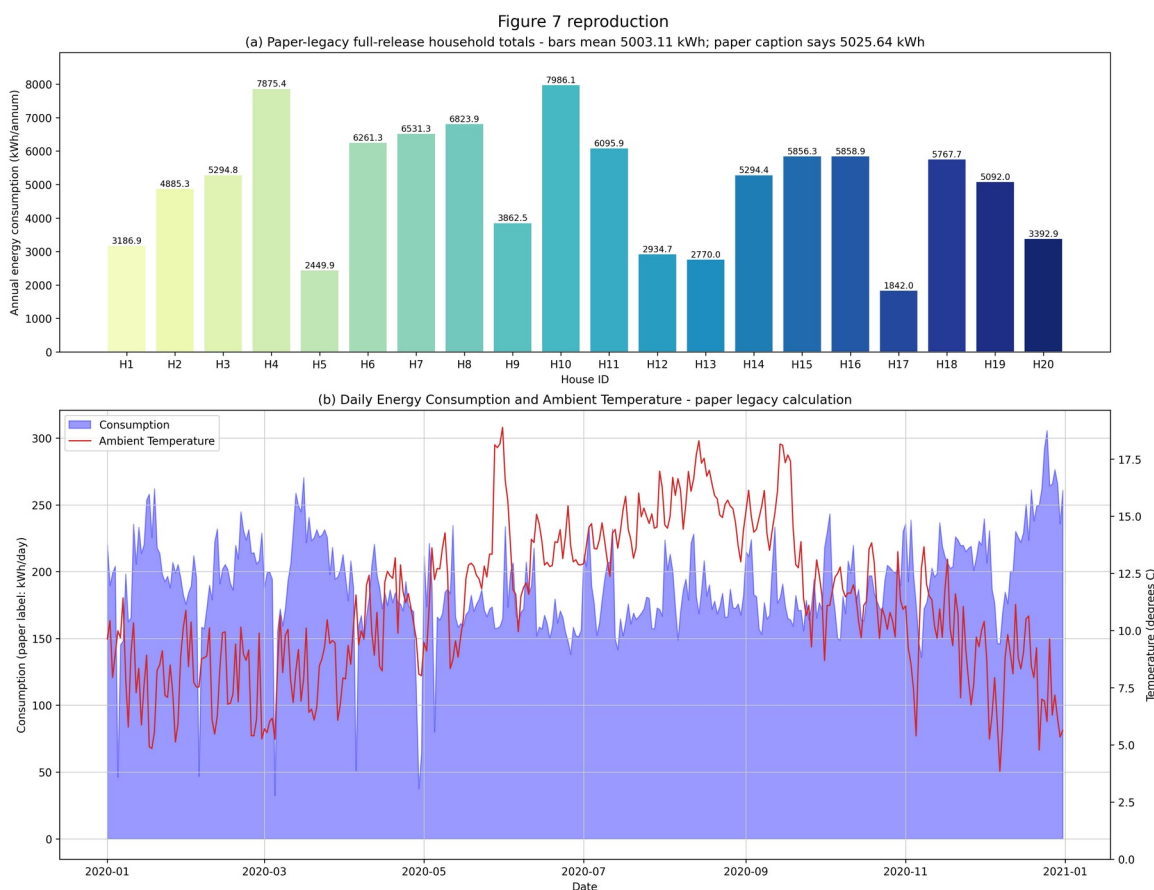


圖 7 照作者原本的算法重畫的 Figure 7。下圖先保留原程式算出的量，方便核對曲線形狀。

來源：結果檔見 [5]；對照 [1] 第 8 頁 Figure 7。

上圖 20 戶的數字，取到小數點後一位，都和論文的柱狀圖相同。但平均值不同：

我們算出每戶 5,003.11 kWh，論文寫的是 5,025.64 kWh。

另外，多數檔案多了 2021 年元旦的第一個小時，H14 又少了年底資料。所以這裡算的是「檔案裡有記錄的用電量」，不是每戶完整的 2020 年用電量。

Figure 7b 的單位為何需要修正

作者 Figure 7b 的程式節錄

```
energy_plots.py  
  
546 df['Sum'] = df.sum(axis=1)  
  
566 daily_average_h1 = df1['Sum'].groupby(df1['Date'].dt.date).mean()  
  
581 ax1.set_ylabel('Consumption (kWh/day)', fontsize=18)  
  
輸入是 Wh 欄位，上面按日取 mean，縱軸卻寫成 kWh/day -
```

圖 8 作者程式的關鍵片段。程式是按日取平均（mean），縱軸卻標成 kWh/day。

來源：官方 energy_plots.py，第 566 與 581 行。

程式先把 20 戶每一分鐘的 Consumption (Wh) 加起來，再對同一天取平均。這樣得到的是「社區每分鐘能量的一日平均」，不是「每天的用電量」。正確的做法是照真正的時間標籤，把一天的 Wh 全部加總，再除以 1,000。如果一天剛好有 1,440 筆完整紀錄，原本的平均值要乘以 1.44 才會變成 kWh / 日。

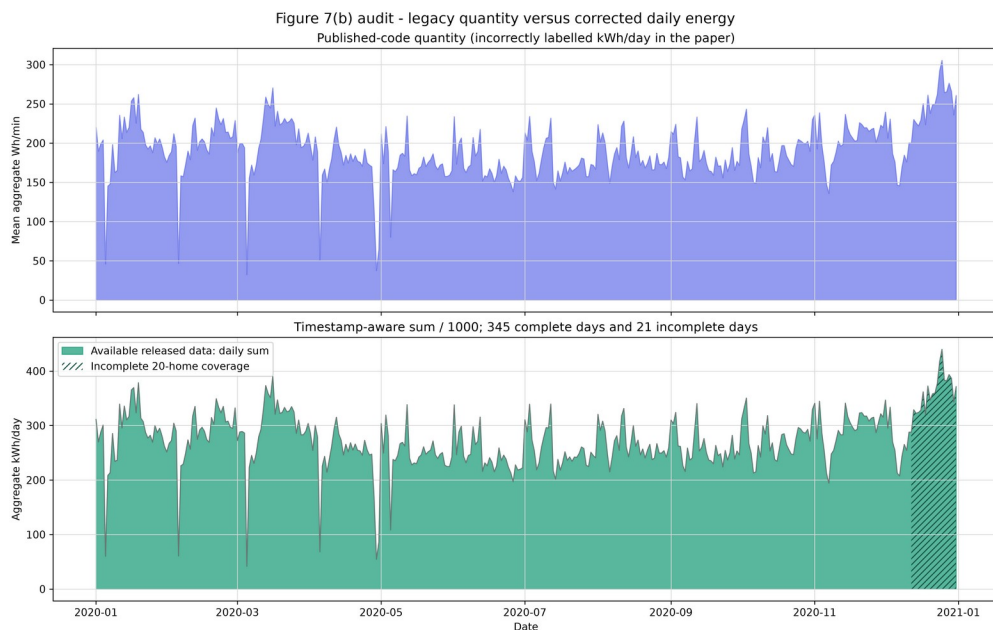


圖 9 原本算法和修正單位後的比較。綠色是當天公開資料的加總；斜線標出年底不滿 20 戶的那一段。

來源：結果檔見 [5]；全年 366 天有 345 天完整。1 月 1 日和 12 月 12–31 日資料不完整，我們沒有補造缺少的值。

這裡的相關係數是比較「社區用電」和「當天平均氣溫」。原本算法約 -0.314 ；依時間標籤加總後約 -0.311 ；只保留 20 戶、1,440 分鐘都有正常數值的 345 天後約 -0.234 。差異來自時間對齊與樣本選擇，單位換算本身不會改變相關係數。

資料論文 Figure 8 有太陽能住戶的能量流

四張雷達圖分別是 PV 發電、住戶用電、送回電網和向電網買電。每一條放射線是一戶，同一戶在四張圖上的位置固定，所以看得出太陽能大小差不多的住宅，為什麼買電的量還是差很多。

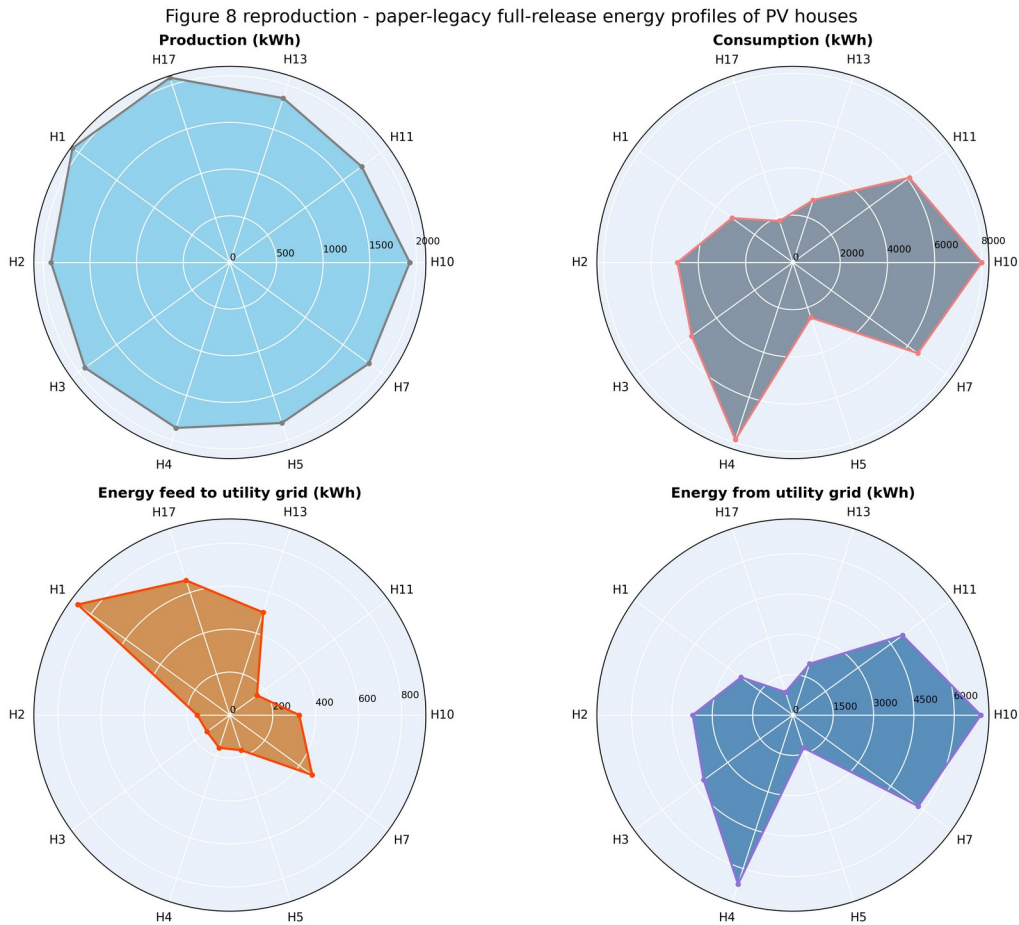


圖 10 我們重畫的 Figure 8。四個量都由 10 戶 PV 住宅的 Wh 檔加總並換成 kWh。

來源：結果檔見 [5]；對照 [1] 第 9 頁 Figure 8。

H4 的 PV 發電約 1,864.89 kWh，用電卻高達 7,875.43 kWh，向電網買了約 6,609.86 kWh，送回電網只有約 160.62 kWh。這和作者說 H4「用電多、買電多、送出去少」一致。10 戶的 PV 發電實際落在約 1,748 到 2,084 kWh 之間，不像正文說的全部介於 1,800 到 2,000 kWh。這裡的加總範圍和公開檔一樣，方便跟原圖核對。

資料論文 Figure 9 H4 一週的電池運作

這張圖把 H4 在 2020 年 12 月 7 日到 13 日的七個訊號疊在一起。從上到下依序是 PV、用電、買電、送回電網、充電、放電和 SoC，對照著看，就能看出電池什麼時候在存電、什麼時候在放電，以及太陽能不夠時電網補了多少。

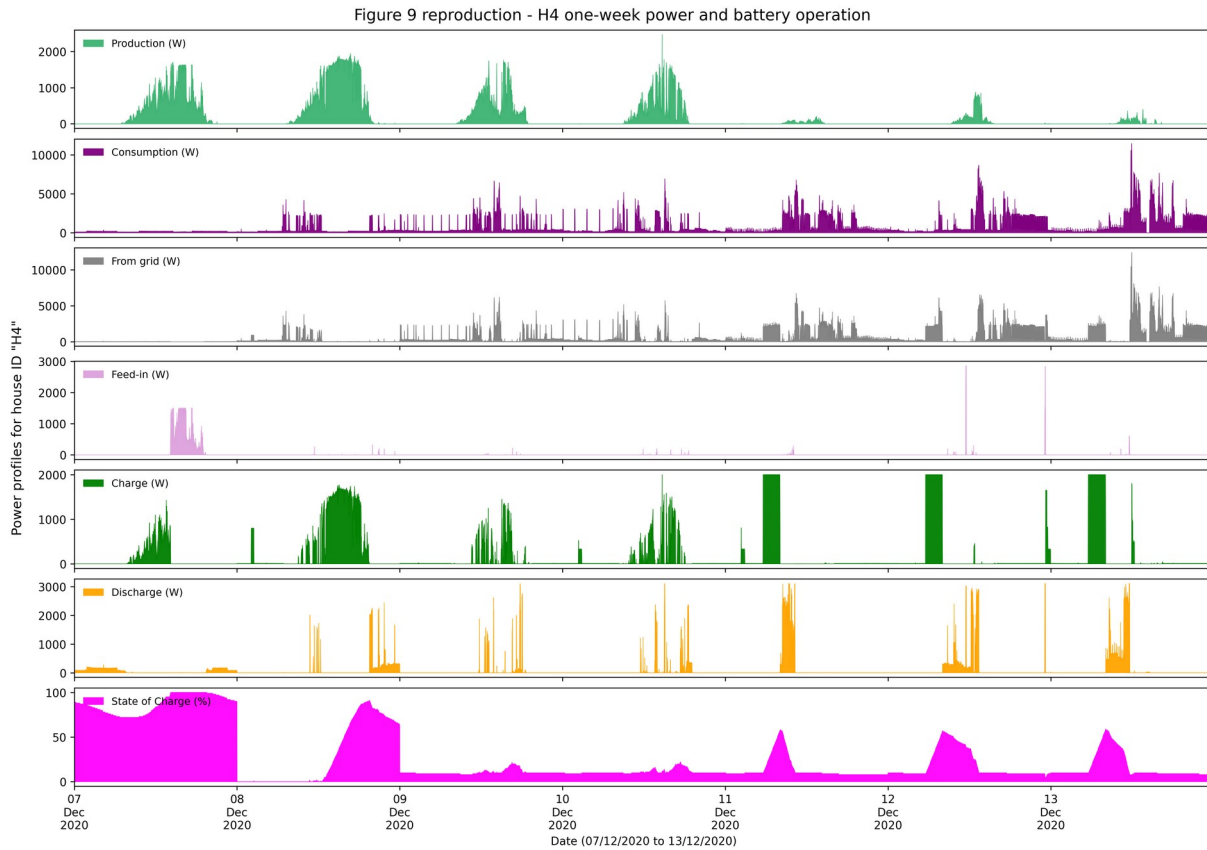


圖 11 我們重畫的 Figure 9。共 10,080 個一分鐘的時間點，這一週沒有缺任何一分鐘。

來源：H4_Wh.csv；結果檔見 [5]，對照 [1] 第 10 頁 Figure 9。

12 月 11 日到 13 日的 PV 很少，那三天的買電量分別約 25.56、25.99、34.90 kWh，和作者說日照不足時會多買電的解釋一致。把所有能量流放回收支式檢查：供給端是買電、PV 和電池放電，需求端是用電、電池充電和送回電網。這一週兩邊差最多的一分鐘只差 0.43 Wh。

日期也有一處需要更正：正文說明顯送回電網的日子是 12 月 8 日，但公開資料和圖上最主要的送電落在 12 月 7 日，約 4.09 kWh；12 月 8 日只有約 0.023 kWh。要注意，H4 整年 W 和 Wh 對不齊，跟這一週 Wh 內部的能量收支可以算得通，是兩件不同層次的事。

資料論文 Figure 10 電壓不平衡

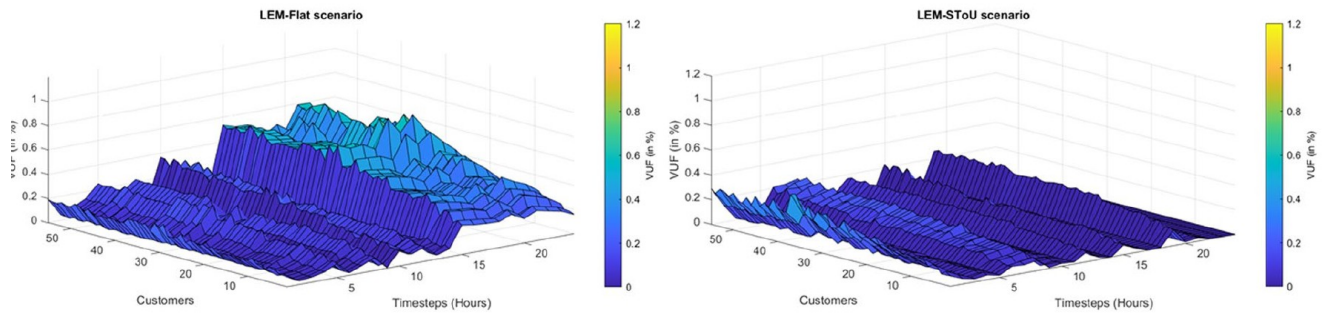


圖 12 資料論文 Figure 10 原圖。左為冬季日的 LEM-Flat，右為夏季日的 LEM-SToU。

來源：Trivedi 等人 [1] 第 10 頁；重刊自 Saif 等人 [4] Figure 13。

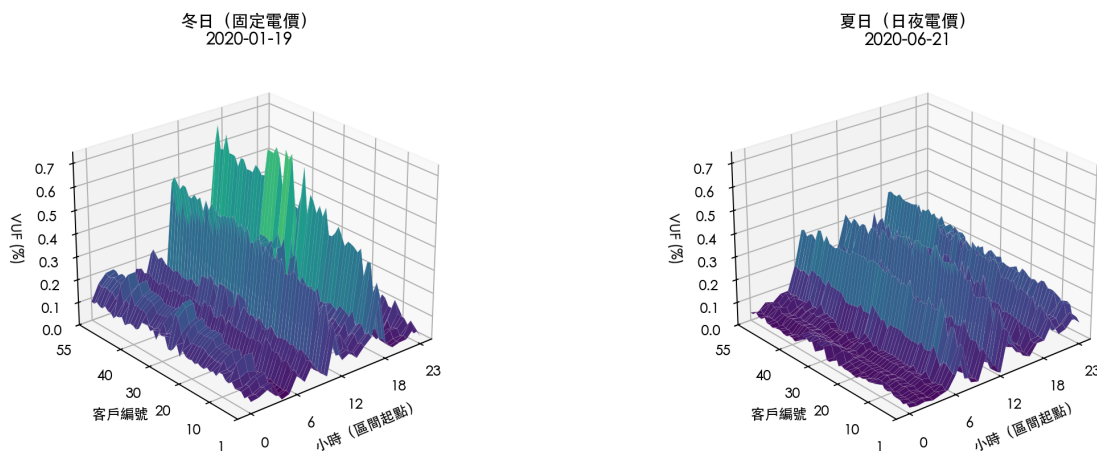
Figure 10 已經進到電網模擬的層次。兩個橫向的軸是用戶節點和一天中的小時，高度和顏色代表電壓不平衡率 VUF。VUF 可以理解成三相電壓偏離平衡的程度，通常用負序電壓和正序電壓的大小比值表示，數值越高代表三相越不平衡。LEM 是地方電力市場，Flat 是固定電價，SToU 是固定時段的時間電價，原研究使用日夜兩段。

左右兩張圖的電價和季節都不一樣，所以不能只比這兩張曲面，就把差異全部算到電價頭上。原研究還把住戶資料放進一個有 55 個單相用戶的低壓配電網，每個節點的電壓是市場交易和電網計算一起決定的。

原圖目前仍無法逐點重現，因為缺少作者修改後的電網模型、55 戶分配規則與完整市場程式。下方是用公開 IEEE 低壓配電網與 StoreNet 資料做的獨立模擬：先排電池充放電，再算三相電壓。左圖最高 VUF 約 0.711%，右圖約 0.390%。這能展示類似的分析，但不是原圖的逐點復現。住戶分配與部分電網設定使用本地假設，完整結果見本包的圖 10 電壓不平衡模擬。

資料驗證這一階段的結論是：Figure 5、Figure 7 論文原本的算法、Figure 8 和 Figure 9 都能用公開資料重算；Figure 6 確認大部分住戶一致，但 H4 是例外；Figure 10 可以說明它在問什麼、怎麼做，但要真正重現需要額外的電網資料。這也劃出了這批資料後面能回答哪些問題。

類似 Fig. 10 的模擬：冬、夏日電壓不平衡



獨立模擬，非原論文逐點復現。55 個客戶由公開曲線循環配置；兩圖共用相同刻度。

第二部分 方法論文與獨立復現

MILP 的問題與基本設定

Bahloul 等人的方法論文 [2] 問的是：同樣一批太陽能電池，交給聚合商統一安排，會比每戶自己操作省多少電費？如果目標換成降低尖峰，又要犧牲多少收益？MILP 同時決定每一條電力流的大小，以及電池這個時段能不能充電或放電；後者用 0 / 1 變數表示。

表 4 MILP 的參數與設定

設定	我們採用的值	意思
住戶與 PV	20 戶 / 10 戶 PV	依公開資料；原文描述為 20 戶 / 9 PV
電池容量	每戶 10 kWh	可使用 SoC 範圍對應 1 至 9 kWh
充電及放電上限	各 3.3 kW	限制的是電池 DC 側的功率
電池充電 / 放電效率	各 0.95	電網至電池再回負載的往返效率為 0.9025
PV 至 AC / PV 至電池轉換	各 0.95	主要設定把公開的 PV 資料當成 DC 側的量
SoC 範圍	10% 至 90%	避免超過設定的可用電量
起始 / 結束 SoC	均為 10%	一天開始及結束都回到 1 kWh
分享損失 ξ	0.07	分享出去的電扣 7%；當成一個比例
額外自放電功率	0 kW	沒有另外加固定的自放電
夜間 / 日間電價	0.091 / 0.194 EUR/kWh	日間為 10:00 至 22:00
饋網價格	0	模型裡送回電網沒有收入
排程長度及解析度	24 小時	代表日 30 分鐘；月度 60 分鐘

四個 0.95 不能全部乘在一起當成每條路徑的效率，電只有經過哪個轉換才扣哪個損失。表中的 ξ 在原論文附錄寫成 7% / day，公式裡卻是當分享損失用。我們照公式把它當成 7% 的比例，另外做一組 $\xi = 0$ 的敏感度分析，把原文這個不清楚的地方保留下來。

依據：[2] 第 5 至 6 頁模型、第 10 頁附錄；src/storenet_config.m 與 src/solve_storenet.m。

電力流向與電池電量怎麼計算

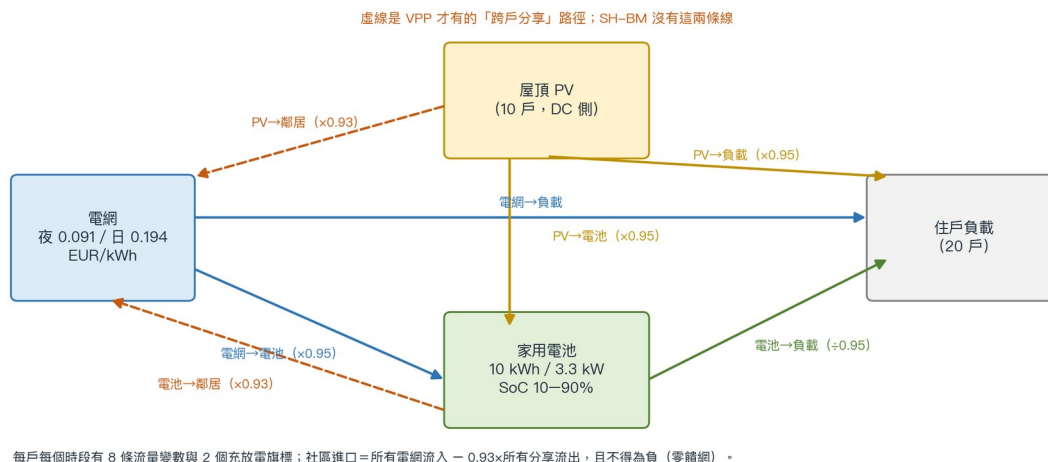


圖 13 單一住戶的供電路徑與住戶之間的分享。模型每個時段都會檢查每條路徑的能量收支。

來源：依我們的 MILP 整理；原模型見 [2] 式 1 至 15。

拿電網充電當例子：電網送出 1 kWh，經過 95% 的充電轉換後，電池只多了 0.95 kWh；這 0.95 kWh 再經過 95% 的放電轉換，最後只能供給負載 0.9025 kWh。PV 直接供負載、PV 充電和住戶之間的分享，也都各自照走的路徑算損失。

$$E_{k+1} = E_k + (C_k - D_k) \Delta t$$

E 是電池裡的能量，單位 kWh；C、D 是換算到電池 DC 側的充電和放電功率，單位 kW。半小時一格的排程 $\Delta t = 0.5$ 小時。這兩個功率各自已經扣過轉換損失，所以更新電量時不會再乘一次效率。

$$1 \leq E_k \leq 9 \text{ kWh} \quad E_0 = E_T = 1 \text{ kWh}$$

規定一天結束時電池還要有 1 kWh，是為了避免策略把一開始的電一次用光，然後把這些「免費的電」算成每天都拿得到的節省。每一天各自求解，今天剩下的電不會隨便帶到明天。

其他限制還有：PV 分出去的量不能超過當時能發的電、住戶的用電一定要被供應、充電和放電各有功率上限，而且同一顆電池不能同時充電又放電。整個社區向電網的淨買電不能是負的；多出來的 PV 可以放棄不用。這是一個把整個社區加總來看的能量模型，沒有算線路電流、變壓器負載或節點電壓。

五種策略各自在做什麼

五種策略用的設備參數和能量限制都一樣，差別只在希望求解器優先改善哪一件事。SH-BM 另外禁止住戶之間分享；其他策略都可以使用社區內的共同資源。

表 5 五種最佳化策略

策略	優先目標	排程上的意思
SH-BM	各戶電費最低	各自利用夜間低價與自家 PV，不與鄰戶分享
VPP-BM	整個社區電費最低	集中安排電池與分享，使社區總帳單最低
PS	全天進口尖峰最低	把一天最高的社區購電功率壓低
PSDT	日間進口尖峰最低	只壓 10:00 至 22:00；夜間尖峰不在主目標內
LL	最大與最小進口的差最小	讓一天的購電曲線更平

例如 VPP-BM 會在便宜的時段多買一些電存進電池，再用來抵掉貴的時段的買電。PS 只要能降低全天的最高功率，就算電費比較高也接受。PSDT 不管夜間的尖峰，所以晚上還是可能集中充電。LL 在意的是整天曲線的高低差，不只是最高點。

同一個主目標常常有很多個排程都同樣最好。為了讓比較有固定的規則，我們讓以電費為目標的策略先把電費降到最低，再減少不必要的電池充放電量；PS、PSDT 和 LL 則先完成各自的主目標，再降電費，最後才減少充放電量。後面的目標只能在前一個目標的最佳值（加上很小的誤差範圍）之內調整，不能反過來犧牲已經固定的主目標。

SB-SC 是現場原本就裝著的商用自用控制器，不是我們另外寫的第六種 MILP。作者和我們都只拿得到它實際運作的量測結果。我們直接用公開的買電、送回電網和電池紀錄來算，代表同一個場域在代表日實際發生的情況。

原本的公式還有幾個地方是我們自己決定怎麼實作的：SH-BM 的式 16 補回 PV 自用對電費的貢獻；SoC 改寫成以 kWh 計的遞迴式；嚴格不等式改成 MILP 能處理的非嚴格不等式。這些都有記錄下來，所以我們的結果是在講清楚的假設下做的獨立復現。

代表日與省錢率的比較基準

方法論文用的是 2019 年 7 月到 2020 年 6 月的內部資料，公開檔則是 2020 年整年，兩邊只有 2020 年上半年重疊。作者也沒有公布 Figure 5 用的是哪一天，或原本 9 戶 PV 是哪幾戶，所以不能期待每個數字都一模一樣。

我們的代表日定為 2020 年 8 月 24 日。做法是先把原 Figure 5 的負載和 PV 曲線數位化，再拿這兩條曲線去公開資料裡找形狀最像、而且資料品質過關的日子。345 天有完整的半小時資料，其中 87 天通過嚴格的品質檢查；挑日子的過程完全沒有用到最佳化後的省錢率。這是公開資料上的替代代表日，不是作者原本用的那一天。

表 6 兩種省錢率用的比較基準不同

基準	代表日電費	比較時回答的問題
原始負載 無 PV 無電池	36.6712 EUR	PV 加電池策略合計省了多少
PV 自用 無電池	33.2971 EUR	相對已有 PV 的住宅，電池控制再省多少

$$\text{省錢率} = (\text{基準電費} - \text{策略電費}) \div \text{基準電費} \times 100\%$$

和論文的 Figure 5、Figure 6、Table I 對照時，本文用第一種基準。代表日 VPP-BM 的電費是 19.5580 EUR，照論文的定義算，省錢率是 46.6664%；改用「已經有 PV」的基準來算，則是 41.2621%。兩個數字都對，只是回答的問題不同。

月度分析每個月取 1、2、15、16 日，共 48 個固定的日子，用一小時一格。其中 3 天因為資料不完整被排除，另外有 1 個 LL 的案例求解失敗；成功的日子和策略組合共 224 個。本文的月度省錢率是先把有效日的電費加總再算比例；尖峰則是各天尖峰的平均，不會拿「平均曲線的尖峰」來混用。

資料與規則：ADR-001、B2022_REPRODUCTION_CONTRACT.md；結果檔見 [6]。

方法論文 Figure 5 代表日原圖

原圖比較五種最佳化策略和現場的 SB-SC。藍線是負載，紅線是向電網買的電，綠線是電池功率，黃線是 PV。先看紅線的最高點，再看每一格的省錢率，就能看出省電費和降尖峰不一定是同一個方向。

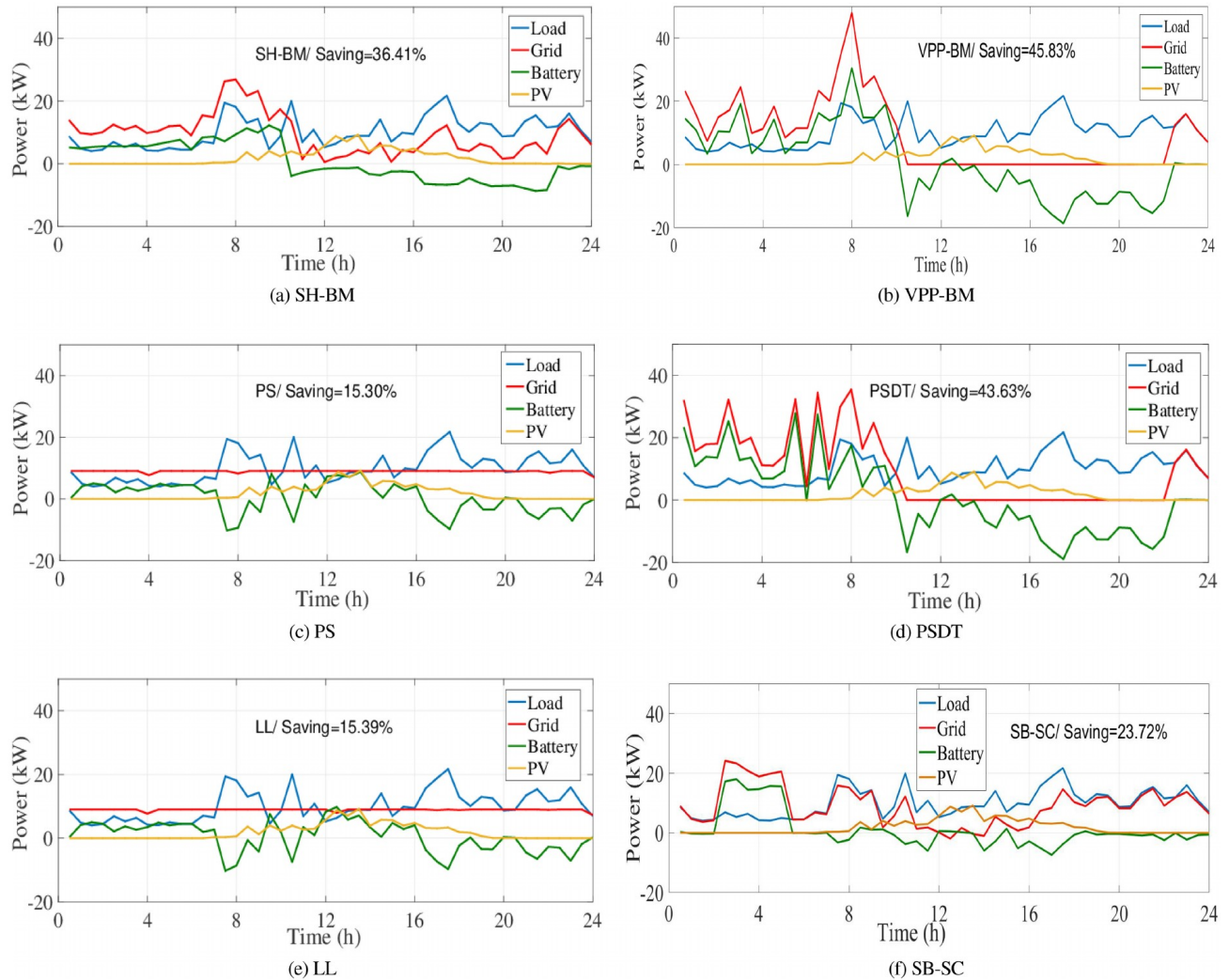


圖 14 Bahloul 等人的 Figure 5 原圖，30 分鐘一格，用的日期沒有公開。

來源：方法論文 [2] 第 7 頁 Figure 5。

VPP-BM 的省錢率最高，PS 和 LL 的買電曲線比較平。SH-BM、VPP-BM 和 PSDT 都看得到晚上在充電，但它們在白天和晚上的尖峰之間怎麼取捨並不一樣。下一頁用同樣的排列順序，方便直接比較。

方法論文 Figure 5 我們的代表日結果

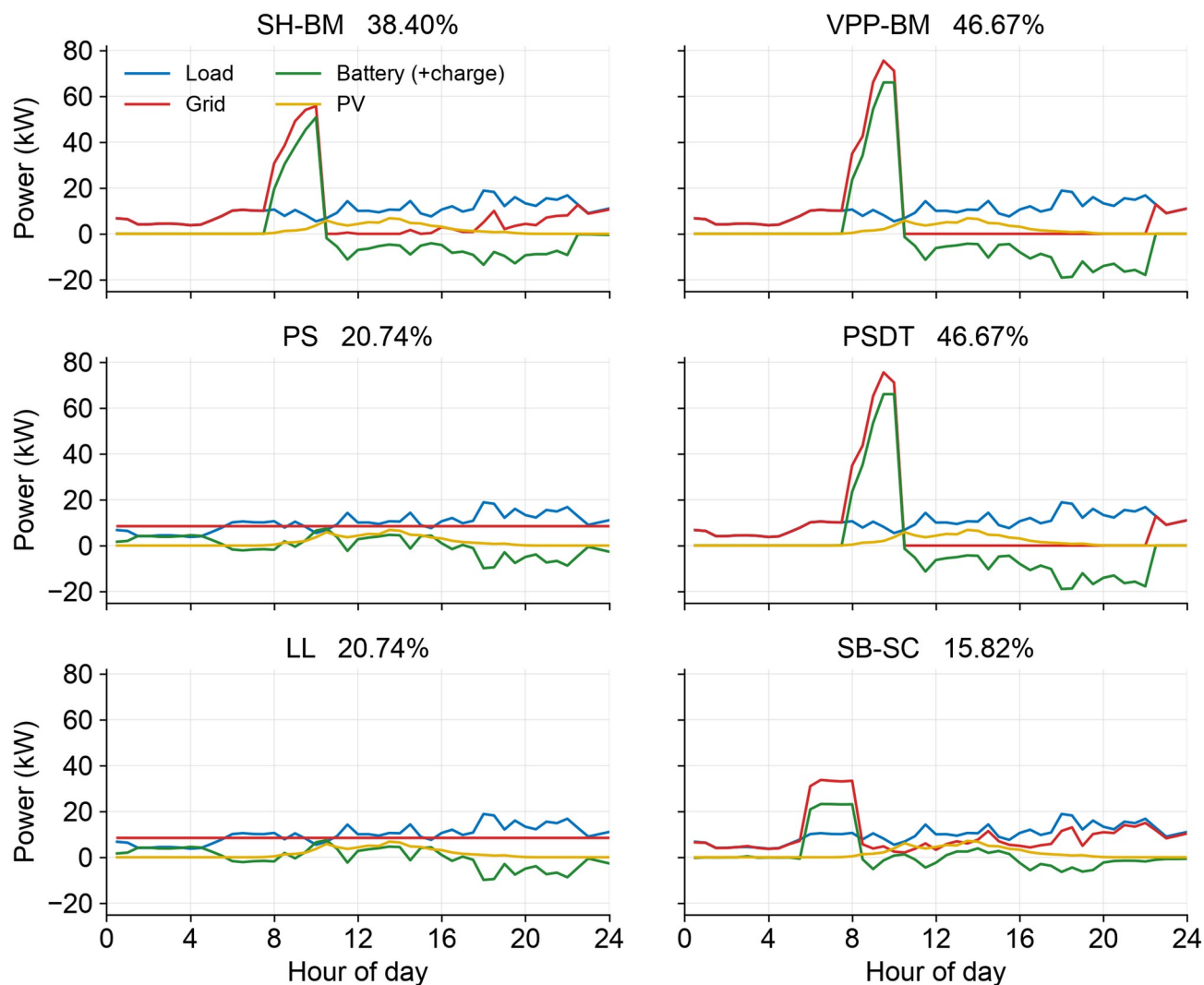


圖 15 我們的 2020 年 8 月 24 日結果。配色與原圖一致；綠線以充電為正。

來源：由結果檔 figure5_profiles.csv 重畫；省錢率取自 typical_metrics.csv [6]。

用公開資料算，VPP-BM 的電費最低，但紅線在早上出現一個 75.50 kW 的新尖峰，遠高於原始負載的 18.87 kW。PS 和 LL 把尖峰降到約 8.50 kW，但省錢率也比較低。PSDT 已經能把白天的尖峰壓到接近零，所以照我們的取捨規則，它最後選到的電費和 VPP-BM 一樣低。

原圖和我們的圖縱軸範圍不一樣。我們的六格共用 -25 到約 80 kW 的範圍，避免每格各自縮放而看不出尖峰的差別。曲線上的點標在每個半小時區間的結束時間；SB-SC 用的是公開的實測資料，其他五個是我們的模型結果。

代表日數字對照與差異解讀

表 7 方法論文 Figure 5 的數值對照

策略	原圖省錢率 %	我們的省錢率 %	差異 百分點	我們的尖峰 kW
SH-BM	36.41	38.40	+1.99	55.68
VPP-BM	45.83	46.67	+0.84	75.50
PS	15.30	20.74	+5.44	8.50
PSDT	43.63	46.67	+3.04	75.50
LL	15.39	20.74	+5.35	8.50
SB-SC	23.72	15.82	-7.90	33.72

VPP-BM 的省錢率和原圖只差約 0.84 個百分點。這是模型行為接近原研究的一個證據，但不代表日期、住戶和原始輸入都相同。PS、PSDT 和 LL 的差異比較大，可能和替代日期，以及我們把電費放在第二順位的規則有關；原作者的完整排程和取捨規則沒有公開，沒辦法確定是哪一個原因。

我們的 VPP-BM 尖峰甚至比 SH-BM 還高，和原文說的「集中調度比各戶自己控制更能降尖峰」不一樣。這說明電費最低並不保證尖峰也比較低；比較的時候要看實際的排程，不能只看策略的名字。

表 8 代表日改變假設後的結果

VPP-BM 另做的情境	省錢率	尖峰 kW
排除 H4	46.41%	67.99
PV 改成當 AC 側的量	47.20%	75.39
分享損失改為 0	47.30%	67.60

換了這些設定，新的尖峰還是很明顯，所以後面的改進不是只在修 H4 這一戶的問題。

方法論文 Figure 6 月度原圖

原 Figure 6 上半部比較每個月的省錢率，下半部比較社區尖峰。作者用 2019 年 7 月到 2020 年 6 月，每個月取 1、2、15、16 日的一小時資料，看代表日的差別在其他月份是不是也一樣。

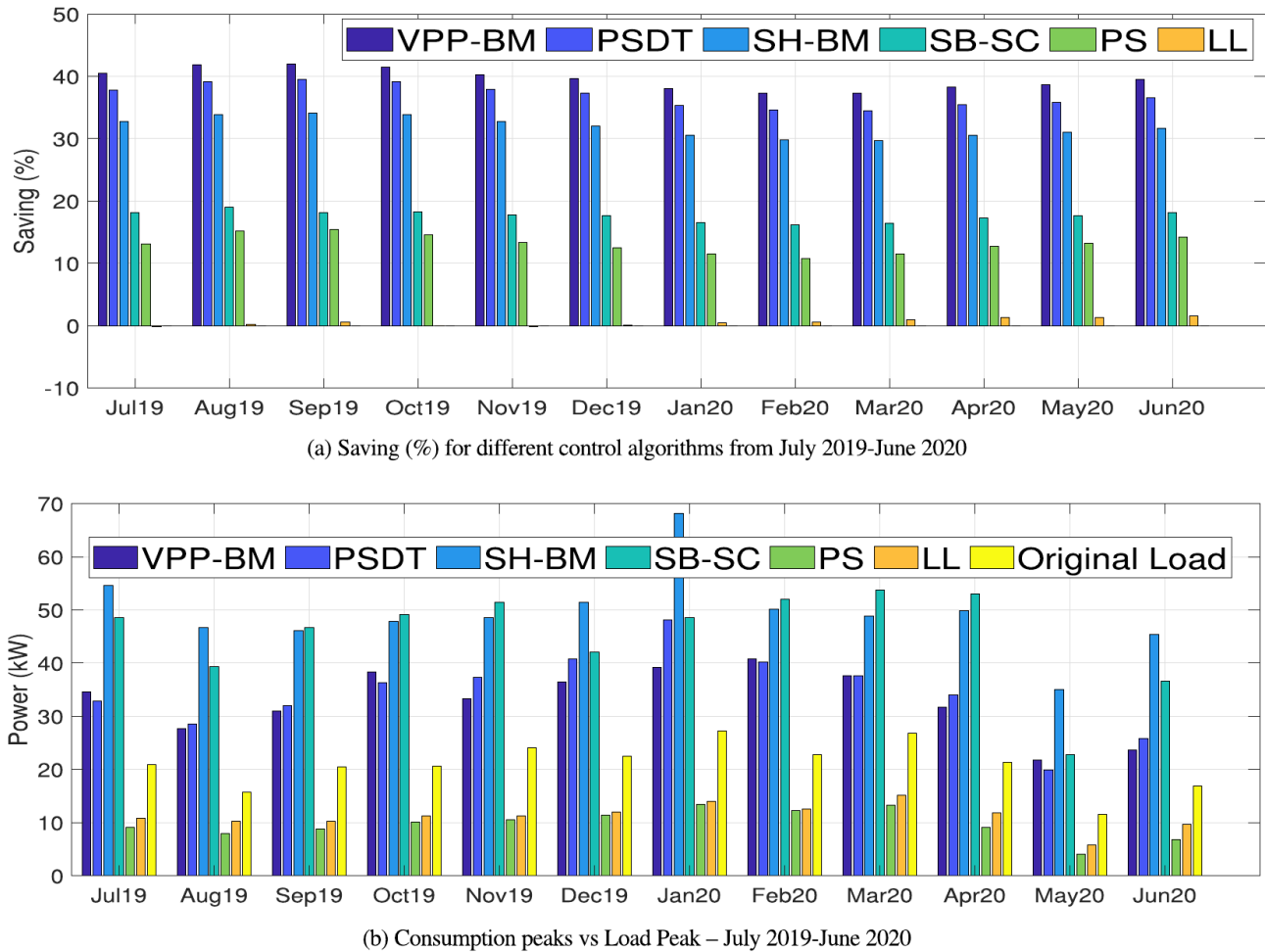


圖 16 方法論文 Figure 6 原圖。SB-SC 是現場實際控制器的實測結果。

來源：Bahloul 等人 [2] 第 8 頁 Figure 6。

原文的省錢率由高到低大致是 VPP-BM、PSDT、SH-BM、SB-SC、PS、LL。PS 和 LL 的尖峰比較低，但省的錢也比較少。這些柱狀圖的期間和下一頁不一樣，同一個位置的柱子不能當成同一天或同一個月來比。

方法論文 Figure 6 我們的月度結果

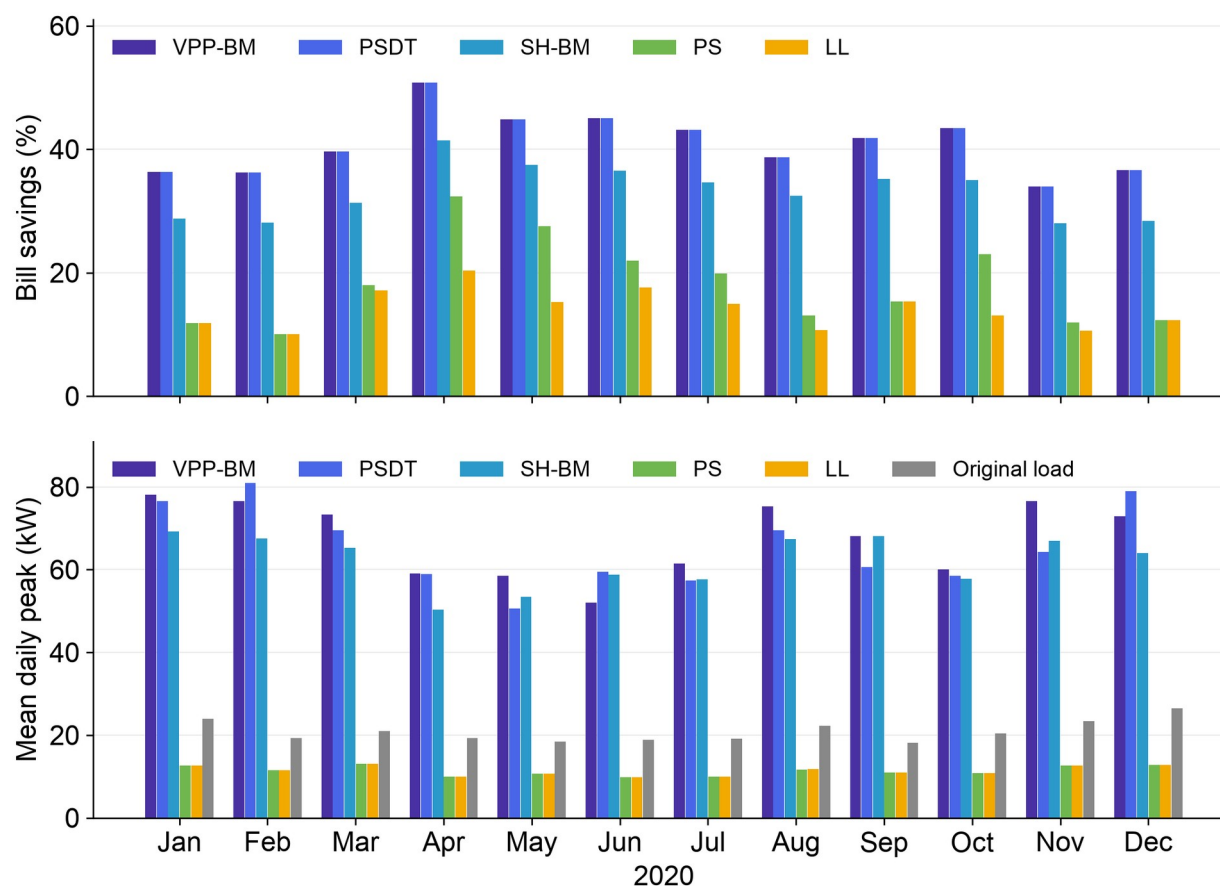


圖 17 我們用 2020 年資料做的月度比較。上圖是各月有效日的總電費比，下圖是各天尖峰的平均。

來源：由結果檔 monthly_metrics.csv 重畫 [6]；五種策略，原始負載當尖峰的參考。

我們的結果也是 VPP-BM 和 PSDT 最省電費，SH-BM 第三。前兩個在一小時一格的設定下最低電費相同；LL 沒有像原圖那樣掉到接近零。這些差異和我們固定的「主目標、其次電費、最後充放電量」規則是一致的，但也受資料期間和負載曲線不同的影響。

48 個固定日裡，1 月 1 日、12 月 15 日和 12 月 16 日因為資料不完整被排除。所以 1 月只有 3 個有效日，12 月只有 2 個；8 月的 LL 另有一個案例失敗，只剩 3 天，其他月份通常是 4 天。這裡的「月度」是固定日期的抽樣，不是把全年每一天都模擬一遍。

VPP-BM 在 45 個可以比較的日子裡，平均尖峰是 67.18 kW；如果先算各月平均再平均一次，會因為每月天數不同而得到另一個數字。本文後面關於改進的跨日比較都用同一組 45 天，避免兩種平均方式混在一起。

方法論文 Figure 7 容量與功率敏感度

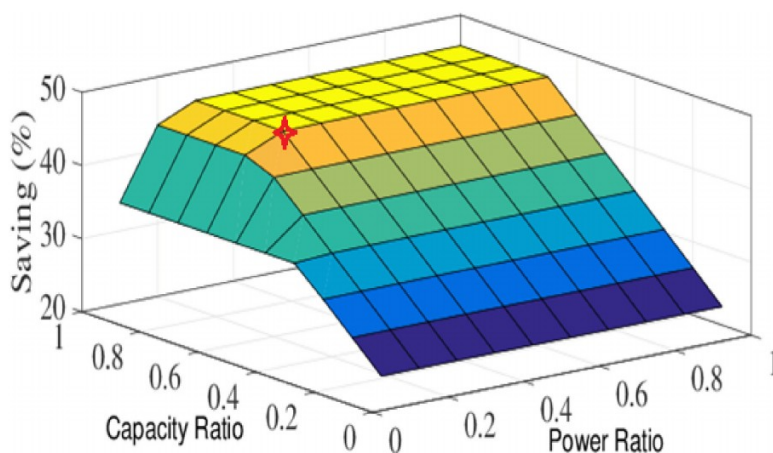


圖 18 方法論文 Figure 7 原圖。作者改變電池容量與功率預算，觀察省錢率。

來源：Bahloul 等人 [2] 第 9 頁 Figure 7。

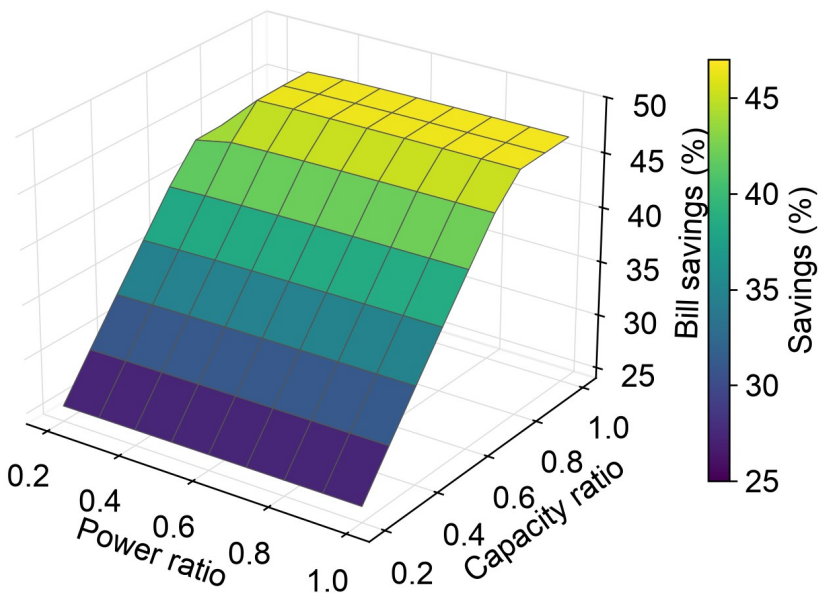


圖 19 我們事先定好的 9×9 網格。容量和功率的比例都從 0.2 到 1.0，每格差 0.1。

來源：結果檔 bahloul_sensitivity_long.csv，共 81 個 VPP-BM 案例 [6]。

兩邊都支持「容量對省錢率的影響比功率大」：我們的結果沿容量方向平均變化約 20.74 個百分點，沿功率方向只有約 1.38 個百分點。不過我們的最大值出現在容量和功率都是 1 的角落，沒有重現作者說的容量 0.7 到 0.8、功率 0.2 到 0.3 那個區域。作者原本的網格沒有公開，所以只比較趨勢方向。

方法論文 Table I 完整對照

TABLE I
BILL SAVING FOR DIFFERENT BATTERIES CAPACITY AND POWER RATIOS

	SH-BM (%)	VPP-BM (%)	PS (%)	PSDT (%)	LL (%)	SB-SC (%)
20% Power	26.76	43.42	15.30	24.17	15.25	--
20% Capacity	16.18	26.45	15.33	18.04	14.86	--
Nominal	36.41	45.85	15.30	43.63	15.39	23.72
Annual Average	31.89	39.358	13.16	36.90	0.54	17.59

圖 20 原 Table I，包括三種電池預算與作者的年度平均列。

來源：Bahloul 等人 [2] 第 9 頁 Table I。

表 9 三種電池預算下的 15 個結果

電池預算	策略	論文 %	我們 %	差異 百分點
額定 100%	SH-BM	36.41	38.40	+1.99
額定 100%	VPP-BM	45.85	46.67	+0.82
額定 100%	PS	15.30	20.74	+5.44
額定 100%	PSDT	43.63	46.67	+3.04
額定 100%	LL	15.39	20.74	+5.35
功率 20%	SH-BM	26.76	31.22	+4.46
功率 20%	VPP-BM	43.42	43.06	-0.36
功率 20%	PS	15.30	20.70	+5.40
功率 20%	PSDT	24.17	43.06	+18.89
功率 20%	LL	15.25	20.70	+5.45
容量 20%	SH-BM	16.18	20.00	+3.82
容量 20%	VPP-BM	26.45	25.46	-0.99
容量 20%	PS	15.33	22.59	+7.26
容量 20%	PSDT	18.04	25.38	+7.34
容量 20%	LL	14.86	15.45	+0.59

VPP-BM 三個點的差異都不到 1 個百分點；PSDT 在功率 20% 時差最多。原表的 Annual Average 用的是 2019 到 2020 年的期間，和我們的取樣日期不同，沒辦法做完全一樣的年度比較；SB-SC 也沒有公開可以調電池預算的控制程式。

作者的總結與留下的問題

作者在 Key Findings、Recommendations 和 Conclusion 裡，把問題拉回用戶、聚合商和電網三方之間的取捨。下面整理原論文的判斷 [2]，再接上我們已經看到的結果。

第一，VPP-BM 最能降低用戶和聚合商的總電費。集中調度可以共用太陽能 and 電池，比各戶自己控制多了很多選擇。這一點在我們的代表日和月度結果裡都成立。

第二，比較省電費的策略可能把原本白天的用電移到晚上。作者指出，就算集中調度在某些比較裡尖峰比各戶自己控制低，還是可能高過原始負載。電池更多、容量更大時，集中充電甚至會做出好幾個新的尖峰。

第三，PS 和 LL 能削峰、能把曲線弄平，但用戶省到的錢比較少。只看電費或只看尖峰，都不一定能照顧到其他參與者的需要。

第四，電池容量和功率要分開看。作者認為省電費主要靠夠大的容量，一部分額定功率可以留給其他服務。我們也重現了容量影響比較大的趨勢，但沒有重現原文說的那個最佳比例區域。

第五，作者建議發展能同時照顧用戶、聚合商和電網的控制策略，以及配套的商業模式或電價。這正是我們改進模型要接著回答的問題：能不能保留大部分的省錢效果，同時不讓電池把社區尖峰推得更高？

所以第一步先保留 VPP-BM 的最低電費目標，再加一條說得清楚的買電上限，這就是 Peak Guard。上限取同一天「有 PV、沒有電池」時的社區尖峰，讓比較有一個明確的基準。後面再檢查這條上限要付多少代價，以及它是不是唯一合理的選擇。

第三部分 控制策略改進

Peak Guard 加上一條尖峰上限

Peak Guard 保留 VPP-BM 的最低電費目標，只多加一條限制：任何時段社區向電網買的電都不能超過當天「PV 自用、沒有電池」情況下的尖峰 P_0 。這個上限來自同一天的負載和 PV，每天都不一樣；不是看到結果以後才挑一個好看的數字。

$$P_{grid,k} \leq P_0$$

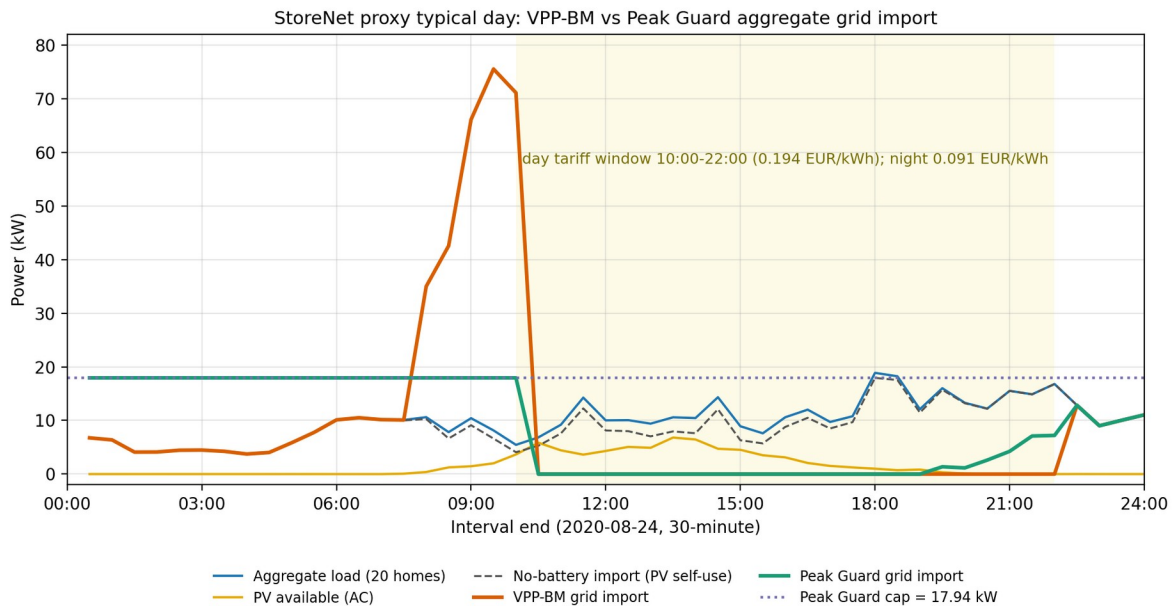


圖 21 代表日的 VPP-BM 與 Peak Guard。虛線是事先定好的 P_0 。

來源：Peak Guard 代表日的配對結果 [7]。

表 10 代表日的效果與電費代價

策略	電費 EUR / 日	尖峰 kW	論文定義的省錢率
VPP-BM	19.5580	75.50	46.67%
Peak Guard	20.5711	17.94	43.90%

尖峰從 75.50 降到 17.94 kW，降了 76.24%；代價是每天多付 1.0131 EUR，省錢率少 2.76 個百分點。 P_0 只是一個好解釋的社區買電基準，不代表電網公司確認過這個功率對每條線路都安全。

Peak Guard 在其他日期的表現

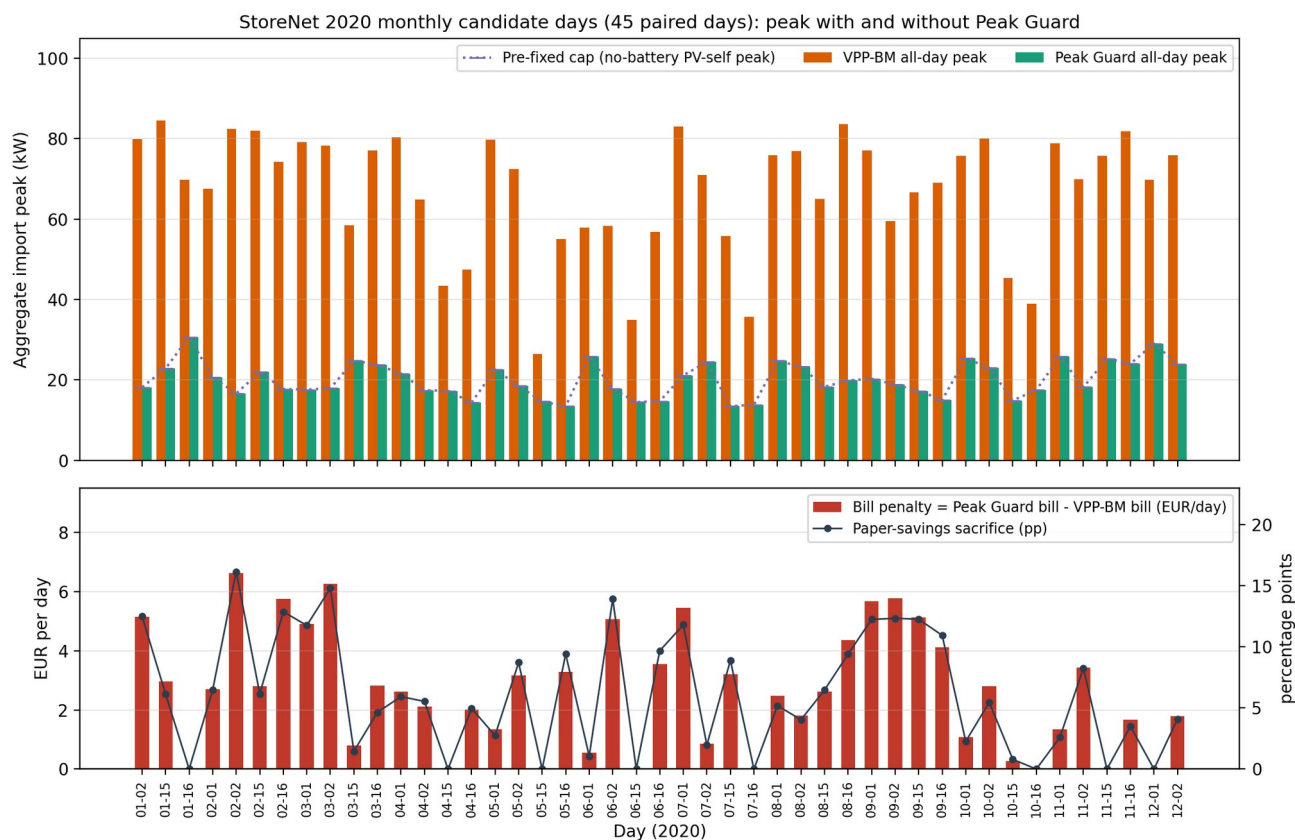


圖 22 StoreNet 45 個有效日的逐日比較。上圖是尖峰，下圖是多付的電費和省錢率的差。

來源：Peak Guard 月度結果檔 paired_comparison.csv [7]。

表 11 相同 45 日的比較

指標	VPP-BM	Peak Guard
平均每日尖峰	67.18 kW	20.06 kW
尖峰超過當日 P_o	45 / 45 日	0 / 45 日
45 日總電費	1,160.55 EUR	1,278.82 EUR
相對 VPP-BM 的電費增加	—	平均 2.63 EUR / 日

45 天都守住了上限，逐日尖峰降幅平均 69.10%。把各天的基準電費和策略電費先加總再算，照論文定義的省錢率少了 6.02 個百分點。37 天要多付電費，另外 8 天在計算誤差範圍內不用多付。

全天的尖峰降低了，白天的買電卻可能增加一些。這不是算錯：電池不能再把充電全部擠到晚上，一天的能量要重新分配。Peak Guard 的目標是守住全天的上限，不是再把白天的尖峰壓到最低。

澳洲資料怎麼轉成同一個控制問題

只在 StoreNet 成立還不夠，所以我們把同一套控制規則放到澳洲 Ausgrid 的住宅負載和屋頂 PV 資料上。這批資料有 300 戶，這次選 2012 年 7 月到 2013 年 6 月，原始欄位每半小時記一次能量。原官方下載頁已失效，本研究使用 Pierre Haessig 的封存副本；來源與雜湊見 [9]。

另存的第二份副本缺少下載連結與取得日期，不能稱為兩條獨立來源。

表 12 Ausgrid 欄位與模型輸入的對應

澳洲欄位	意思	放進模型前的處理
GC	一般住宅用電	與 CL 相加作為負載
CL	獨立計費的控制負載	先檢查有沒有缺列，不能一律當成 0
GG	獨立量測的 PV 發電	已經是逆變器後的 AC 量，不再扣一次 PV 到 AC 的效率
48 個時段值	每半小時的 kWh	除以 0.5 小時，轉成平均 kW

資料論文列出 54 戶資料比較乾淨的住戶。Customer 2 缺了一段 GC 和 GG，先排除，剩下 53 戶。再檢查後發現 Customer 161 在 2013 年 2 月到 3 月有 59 天沒有 CL 這一系列；原本「缺 CL 就當成 0」的轉換規則，分不出「真的沒有這種負載」和「這筆負載沒有被公開」。

為了確認結論不是靠這個假設撈起來的，我們另外做了一組把 161 整戶拿掉的分析，改用 52 戶 / 52 PV，再照每個月的負載和 PV 曲線各挑一個代表日，共 12 天。這個處理是把有疑問的住戶排除，不是幫它補造用電量；這組分析是事後才加的，不能說成一開始就計畫好的。

電池容量、功率、分享損失和日夜電價都沿用 StoreNet 的設定，讓比較的重點放在資料本身的不同。這是把同一套規則搬到另一組曲線上的控制實驗，不代表澳洲住戶當時真正的電費。PV 的量測點則依資料性質調整，避免重複扣效率。

澳洲結果為何讓我們重新思考

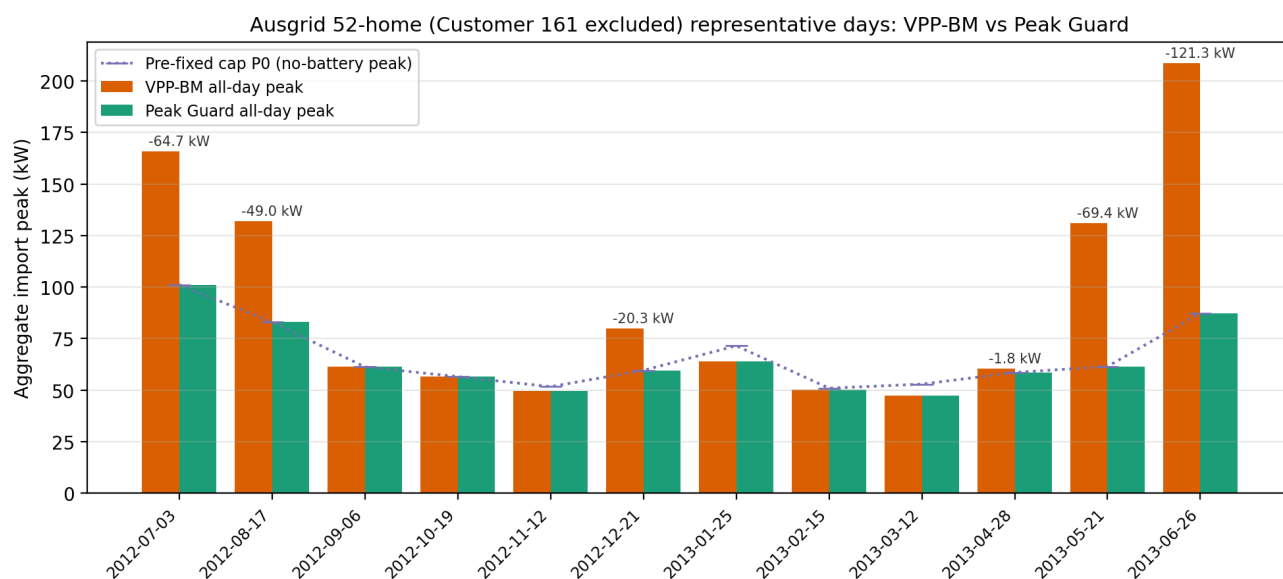


圖 23 排除 Customer 161 後，52 戶的 12 個代表日比較。

來源：排除 Customer 161 後的 Peak Guard 配對結果 [7]。

表 13 兩個資料集的共同效果與不同代價

比較項目	StoreNet 月度	Ausgrid 排除 161
配對日數	45 日	12 日
VPP-BM 形成新尖峰	45 / 45 日	6 / 12 日
Peak Guard 守住 Po	45 / 45 日	12 / 12 日
增加的電費	平均 2.63 EUR / 日	為零（只剩計算誤差）

澳洲的結果最特別的地方不是尖峰降了多少，而是電費幾乎沒有增加。12 天裡最大的電費差只有約 3×10^{-13} EUR，這只是計算誤差。例如 2013 年 6 月 26 日，VPP-BM 的尖峰是 208.58 kW，Peak Guard 把它降回當天的上限 87.32 kW，電費卻還是同樣最低。

這給了一個線索：有些日子的最低電費排程可能不只一種。一種是集中充電，另一種把充電攤開；如果電費一樣，原本的大尖峰就不一定是省電費必須付的代價。不同資料集的負載、PV 比例和曲線形狀，都會影響可以調整的空間；我們沒有把這些因素一個一個分開比較，所以不把差異歸給單一原因。

沿著這個問題，第二版回到 StoreNet：先不指定 Po 上限，直接問在電費同樣最低的所有排程裡，尖峰最低可以到哪裡？

第二版模型先把免費的削峰找出來

新策略叫 VPP-BM+peak，程式裡的名稱是 VPP_BM_PEAK_LEX。它照順序解三個目標：先求出最低電費並固定住，再把全天尖峰降到最低，最後在前兩個條件下減少電池的充放電量。這個順序把「不增加電費」變成明確的條件，不是事後看到兩個電費很接近才下結論。

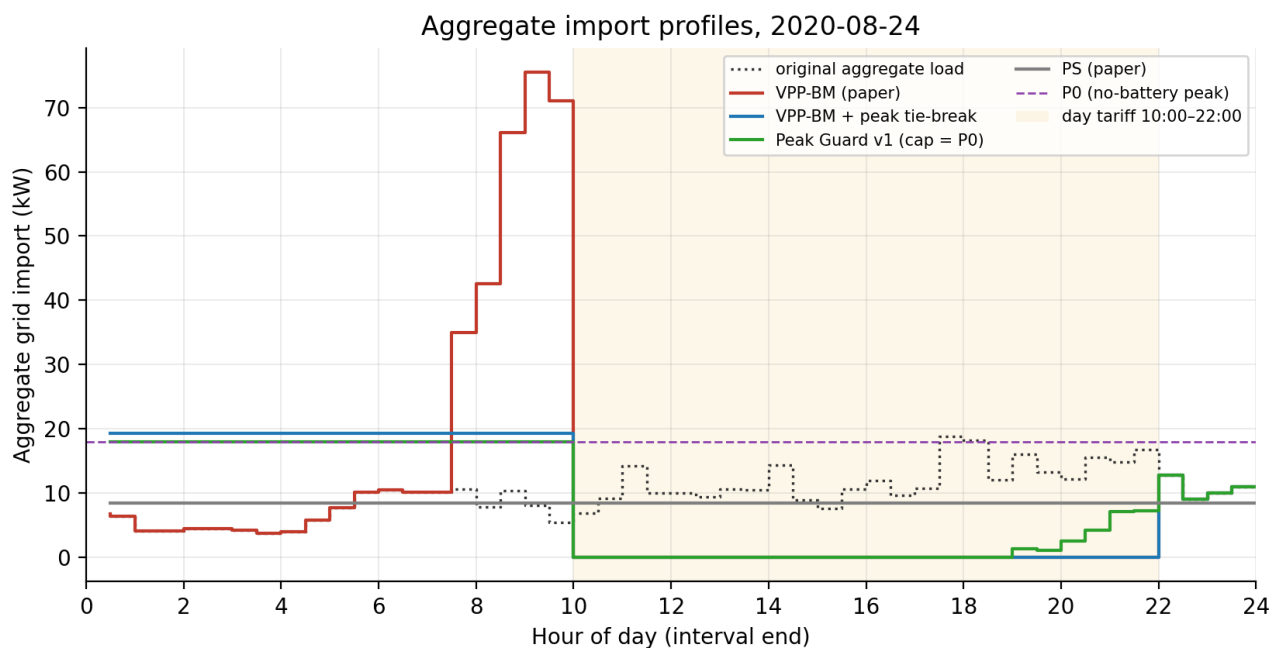


圖 24 同一個代表日的排程。VPP-BM+peak 把夜間充電分散開來，電費維持最低。

來源：第二版代表日結果檔 profiles.csv 與 anchor_comparison.csv [8]。

表 14 先維持最低電費，再降低尖峰

指標	代表日	45 個月度日
VPP-BM 尖峰	75.50 kW	平均 67.18 kW
VPP-BM+peak 尖峰	19.35 kW	平均 23.10 kW
逐日尖峰降幅	74.38%	平均 65.27%
相對原本的最低電費	相同	每天的差都在計算誤差內

晚上每個時段的電價都一樣，所以充電時間有調整的空間。我們的結果證明，原本 VPP-BM 排程的大部分尖峰，都可以在電費同樣最低的排程裡消掉。不過作者的原始輸入和完整的解沒有公開，不能進一步說原論文的每一個尖峰都是同樣的原因。

同帳單的削峰在全年取樣日都成立

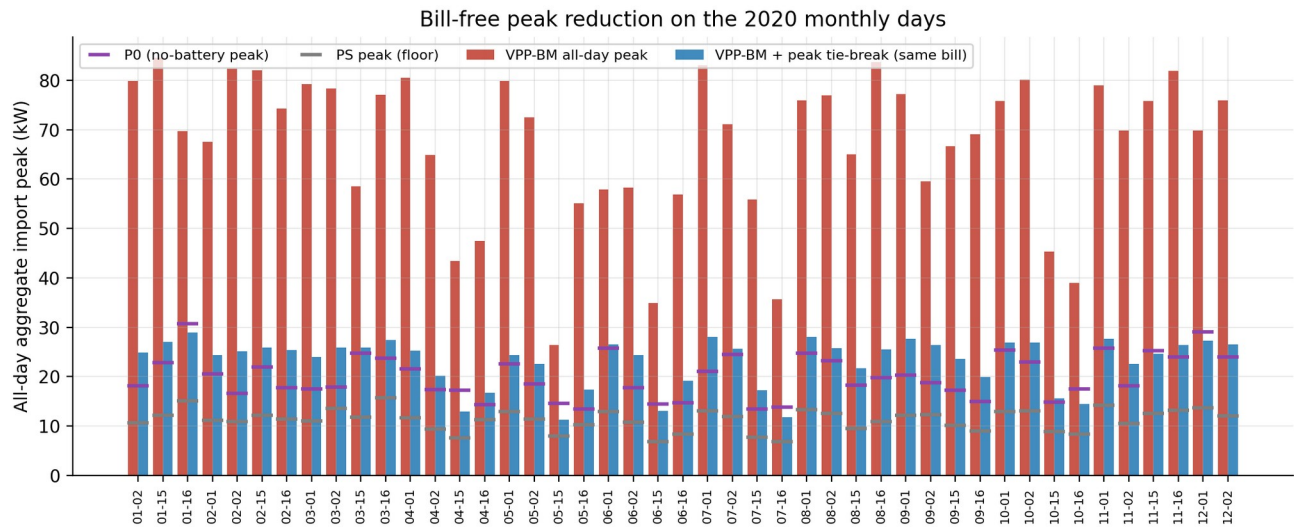


圖 25 45 天的 VPP-BM 與同電費最低尖峰。Po 和 PS 是兩條參考線。

來源：第二版月度 anchor_comparison.csv [8]。

45 天的降幅都是正的，逐日大約在 54.1% 到 71.1% 之間。把電費保留到全部小數位來比，最大的逐日差約 9.95×10^{-14} EUR，真實帳單根本分不出來。這表示這種排程調整在全年的固定取樣日都有用，不是只在 8 月 24 日成立。

不過，同電費的最低尖峰不一定已經低於 Po。代表日還高出 1.41 kW；45 天裡只有 8 天低於或等於 Po。這和 Peak Guard 有 8 天不用多付電費的結果剛好對上：當 Po 落在最低電費排程做得到的範圍內，這條上限就是免費的；要求得更低，就開始要付電費。

表 15 把免費部分與付費部分分開看

模型的要求	代表日尖峰	相對 VPP-BM 電費
只求最低電費 VPP-BM	75.50 kW	基準
電費不變之下再削峰	19.35 kW	0 EUR / 日
守住無電池上限 Peak Guard	17.94 kW	增加 1.0131 EUR / 日
把全天尖峰降到最低 PS	8.50 kW	增加 9.5059 EUR / 日

第一版 Peak Guard 其實已經包含了大部分不用付錢的削峰，只是當時沒有把兩段分開。第二版讓我們能講清楚：哪些改善只是挑了一個更好的最低電費排程，哪些改善才真的要犧牲收益。

再降低尖峰要付多少電費

為了把取捨看完整，我們把上限改成 r 倍的 P_0 ，一個一個求最低電費。 r 越小，要求越嚴。把每個有解的點的尖峰和電費畫出來，就是電費和尖峰的取捨曲線；Peak Guard 就是 $r = 1$ 的那一點。

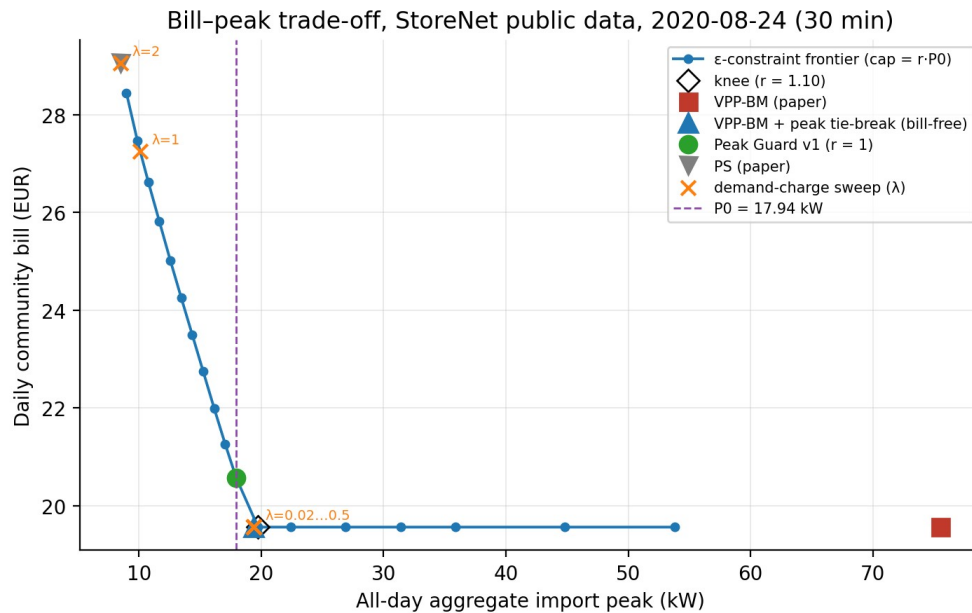


圖 26 代表日的電費與尖峰取捨曲線。水平的那一段代表可以削峰又不用多付電費。

來源：第二版代表日 frontier_daily.csv [8]。

表 16 代表日部分取捨點

上限 r 倍 P_0	尖峰 kW	電費 EUR / 日	增加電費 EUR / 日
1.10	19.73	19.558	0.000
1.00	17.94	20.571	1.013
0.90	16.14	21.989	2.431
0.80	14.35	23.497	3.939
0.60	10.76	26.619	7.061
0.50	8.97	28.446	8.888

代表日把尖峰從同電費的 19.35 kW 壓到 $P_0 = 17.94$ kW，要多付 1.0131 EUR。再往下，在 $r = 0.95$ 到 0.5 之間相鄰的點，每多降 1 kW 尖峰大約要多付每天 0.77 到 1.09 EUR。這是這組資料和電價下的結果，不能當成通用的削峰價格。

月度取捨與不可行的上限

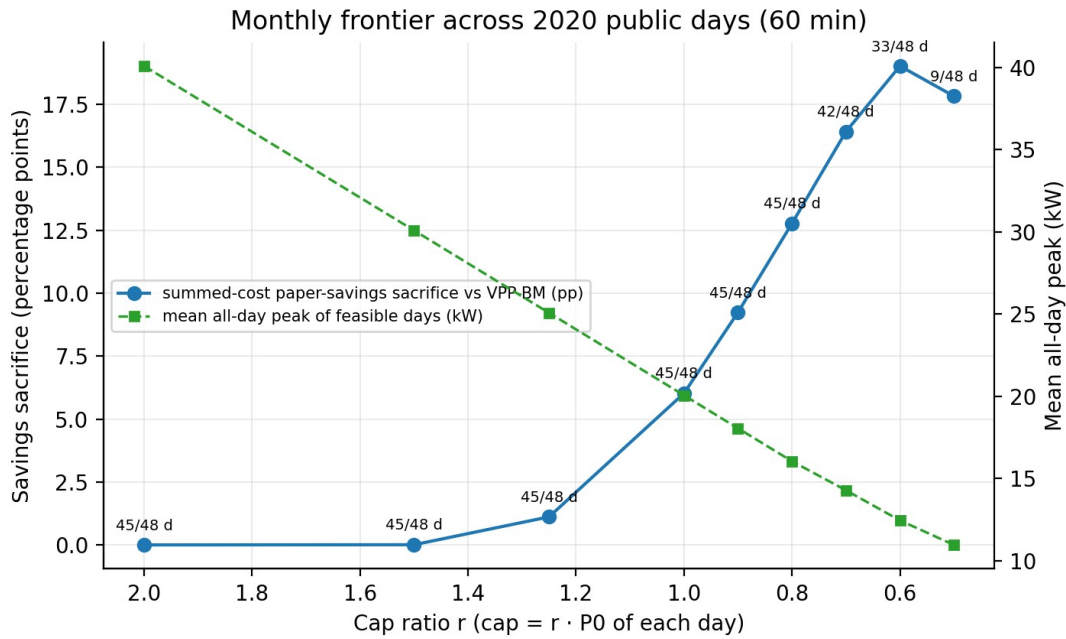


圖 27 月度取捨曲線。上限更低時代價增加，有些日期也開始無解。

來源：第二版月度 frontier_summary.csv [8]。

表 17 月度取捨的樣本與代價

上限 r 倍 P_0	有解的天數	這些天多付的電費合計 EUR	省錢率減少 百分點
1.50	45	0.14	0.01
1.25	45	21.87	1.11
1.00	45	118.27	6.02
0.90	45	181.39	9.24
0.80	45	250.73	12.77
0.70	42	303.02	16.42
0.60	33	274.99	19.02
0.50	9	71.32	17.83

$r=1.25$ 時，45 天裡有 30 天不用多付電費，合計只多付約 21.87 EUR；收緊到 $r=1$ 時，合計變成 118.27 EUR。這一段多了約 96.40 EUR，可見上限訂在哪裡對收益影響很大。

$r \leq 0.7$ 時有解的天數變少，總代價只是那些有解的日子和同一天的 VPP-BM 相減，不能直接和 45 天的總額比大小。第二版月度共 816 個案例：714 個成功、51 個因為上限太低無解、51 個因為資料品質被排除，沒有求解失敗或逾時。

需量電價會讓聚合商選到哪裡

上限型的做法是直接規定尖峰不能超過多少；價格型的做法是照尖峰收費，讓聚合商自己衡量。這裡的需量費是一天最高的買電功率乘以 λ ， λ 的單位是 EUR / kW / 日。最佳化的目標變成原本的度電費加上這筆需量費。

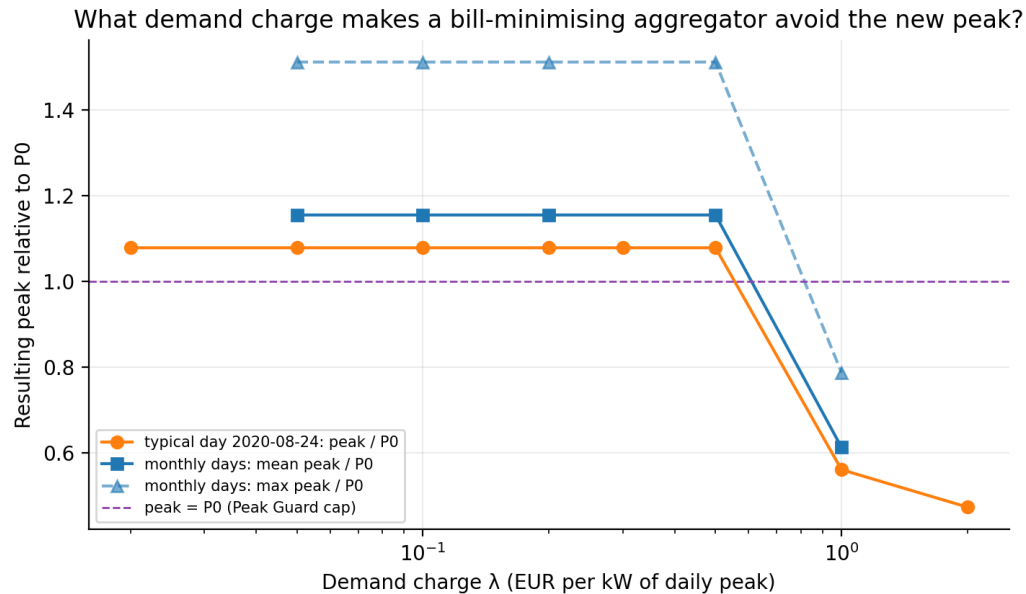


圖 28 需量電價與最後的尖峰。這裡把尖峰除以當天的 P_0 ，方便不同日期互相比較。

來源：第二版代表日與月度 tariff_daily.csv、tariff_summary.csv [8]。

表 18 測試的幾個需量費率與結果

λ EUR / kW / 日	代表日尖峰	代表日增加度電費	45 日尖峰 $\leq P_0$
0.05	19.35 kW	約 0 EUR	8 日
0.50	19.35 kW	約 0 EUR	8 日
1.00	10.07 kW	7.695 EUR	45 日
2.00	8.50 kW	9.506 EUR	月度沒有跑這個費率

代表日測過 $\lambda = 0.02$ 到 0.5，結果都把排程推到接近同電費的最低尖峰；但這不代表所有正的費率、所有電價結構都會這樣。到 $\lambda = 1$ 時，月度 45 天都低於 P_0 ，但平均壓到只剩約 0.61 倍 P_0 ；多付的度電費合計 402.30 EUR，大約是 Peak Guard 的 3.4 倍。

如果目的只是守住某個上限，用價格來做可能會壓得太低、也付得太多。表裡的「增加度電費」不包含另外收的需量費，需量費已經算在求解目標裡。這是按每天尖峰收費的模型，不能直接當成真實電價裡按整月最高需量收費的做法。

用公開電池資料檢查效率假設

主模型沿用論文的充電和放電效率各 95%，相乘是 90.25%。另一個檢查是把公開的 2020 年 20 戶電池放電量和充電量加總，得到約 $30,153.2 / 42,035.5 = 71.7\%$ 的放電對充電比。這個數字混合了待機、自放電、量測位置和資料完整性的影響，不能直接當成電池本身的效率。

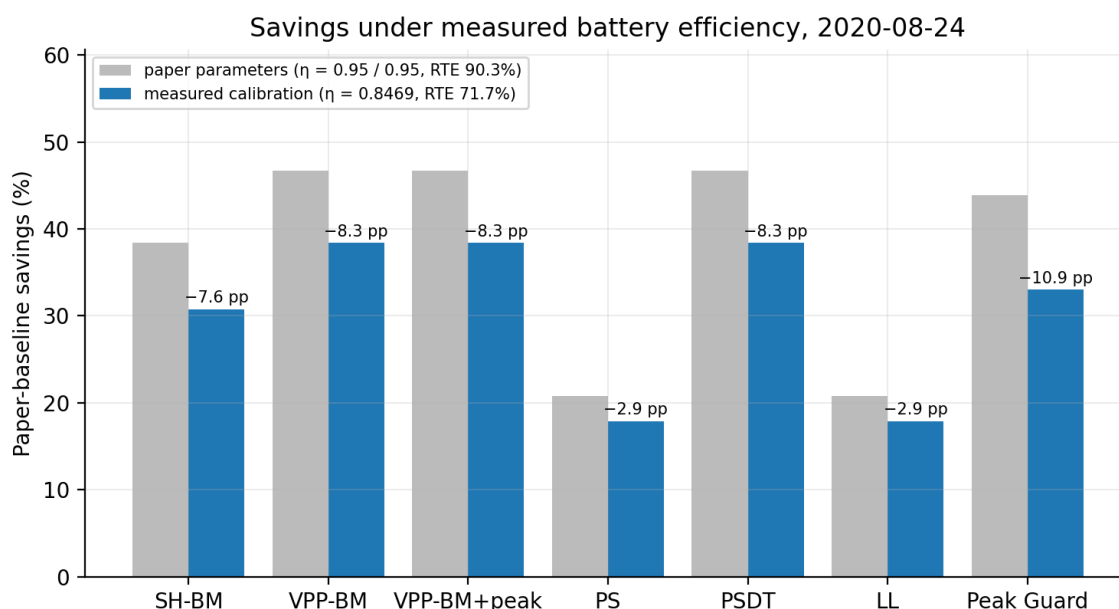


圖 29 代表日改變效率後的結果。把模型的充放電效率都改成 0.8469 後，省錢率下降。

來源：第二版 efficiency_comparison.csv [8]； 0.8469^2 約為 0.7172。

表 19 代表日省錢率對效率的敏感度

策略	論文效率設定	較低效率設定	下降 百分點
SH-BM	38.40%	30.76%	7.64
VPP-BM	46.67%	38.41%	8.26
VPP-BM+peak	46.67%	38.41%	8.26
PS	20.74%	17.89%	2.85
Peak Guard	43.90%	33.02%	10.88

VPP-BM 的省錢率從 46.67% 降到 38.41%；Peak Guard 掉得更多，因為效率變差了還是要守住同一條上限。VPP-BM+peak 在較低的效率下還是找得到同電費、尖峰較低的排程，但最低尖峰也會變高。這一輪只重算代表日，沒有把結果推廣成全年的實際省錢率。

總結與接下來的研究

這個研究先處理資料不可信，再處理控制策略。官方下載的 46 個檔案已經核對過；資料論文的主要圖表都能追到公開檔和程式，但 H4 的 W / Wh 對不齊、Figure 7 的單位和文字數字，以及 Figure 10 缺少電網資料，解讀結果時都要記得。

在這個基礎上，方法論文的五種策略已經用公開資料獨立復現。VPP-BM 最省電費、容量比功率更影響收益這些方向都成立；日期、PV 戶數和取捨規則不同造成的差異，也都清楚列在原圖和我們結果的對照裡。

表 20 改進工作的推進順序

階段	多做了什麼	得到的答案
Peak Guard	在最低電費問題加入 Po 上限	尖峰可控，StoreNet 多數日期要付額外電費
Ausgrid 檢查	處理負載缺列後比較 52 戶	部分削峰在電費不變之下就做得到
第二版模型	先固定電費再削峰，並掃描上限	分清哪些改善免費、哪些要付錢
電價與效率分析	改用需量費目標及較低效率	制度與設備假設都會改變代價

就目前的結果來看，先求最低電費、再加一個削峰目標，是值得保留的控制規則。如果還需要一條明確的買電上限，再照取捨曲線討論願意多付多少電費。代表日和月度取樣都支持這個做法，但還沒有替電網公司決定實際該用哪一個上限。

下一步最直接的是把「事先完全知道負載和 PV」改成實際的預測，用滾動排程檢查收益會少多少、上限會不會被突破。第二版策略也還沒有完整搬到 Ausgrid 做同樣的取捨曲線驗證。如果要進一步談電壓或變壓器的安全，要先拿到沒參加的住戶負載和線路參數。這幾件事都是目前還沒回答的問題，也比繼續追原論文沒公開的輸入、硬要把數字對到一樣更有意義。

附錄 策略算式與 MATLAB 程式

算式與程式怎麼對照

前面的圖回答的是各種策略跑出來有什麼差別，這份附錄則把它們實際在解的問題寫清楚。先列共同的模型，再依序列出五個原策略和我們的改進。讀的時候只要抓住三件事：電從哪裡來、電池能存多少，以及求解器先改善哪一個指標。

表 21 算式與求解順序總覽

策略	共同模型之外的設定	我們的求解順序
SH-BM	禁止住戶之間分享	電費 $B \rightarrow$ 電池吞吐量 Q
VPP-BM	允許住戶之間分享	$B \rightarrow Q$
PS	全天尖峰變數 z	$z \rightarrow B \rightarrow Q$
PSDT	日間尖峰變數 zD	$zD \rightarrow B \rightarrow Q$
LL	最高 z 與最低 m	$z - m \rightarrow B \rightarrow Q$
Peak Guard	每時段進口 $Gt \leq P_0$	$B \rightarrow Q$
第二版 VPP-BM+peak	最低電費之後加入削峰	$B \rightarrow z \rightarrow Q$
上限掃描	$Gt \leq rP_0$ ，逐一改變 r	每個 r 都解 $B \rightarrow Q$
需量費模型	費率 λ 乘全天尖峰	$B + \lambda z \rightarrow Q$

箭頭代表分步驟求解。前一步算完，就把那個最佳值固定住，下一步只能在很小的誤差範圍內調整。下面每一步都保留共同限制和前面已經加進來的限制；星號代表那一步算出來的最佳值。除了需量費模型以外，這些步驟都不是用權重加在一起。

五個策略的主目標來自方法論文 [2]；後面用電費或充放電量來做取捨，以及求解的誤差範圍，是我們為了固定比較規則自己定的。SB-SC 用的是現場量測紀錄，沒有列成第六個最佳化策略。

程式框保留了實際程式的變數名稱和求解順序，省略了儲存求解紀錄的部分；誤差範圍改成直接呼叫同一個函式。它們是方便閱讀的節錄，不是可以各自獨立執行的完整檔案。報告附的 MATLAB 程式包裡是沒有改寫過的原始程式。

共用符號與決策變數

用 i 代表住戶、 t 代表時段， $N = 20$ 。代表日每格 30 分鐘， $T = 48$ 、 $\Delta t = 0.5$ 小時；月度計算每格 1 小時。 $L_{i,t}$ 是負載， $A_{i,t}$ 是轉換前可用的 PV 功率，兩者都是資料輸入，單位 kW。沒有 PV 的住戶 $A_{i,t} = 0$ 。

表 22 八個功率變數與電池狀態

算式記號	MATLAB 變數	意思與量測位置
$pH_{i,t}$	pvToHomeKW	PV 供自家負載，轉換後 AC 功率
$pB_{i,t}$	pvToBatteryKW	PV 充電，轉換後進入電池的 DC 功率
$pS_{i,t}$	pvToGridKW	PV 分享輸出，轉換後、分享損失前 AC 功率
$u_{i,t}$	pvCurtailKW	棄光，尚未使用的 DC 可用功率
$gH_{i,t}$	gridToHomeKW	電網供自家負載的 AC 功率
$gB_{i,t}$	gridToBatteryKW	電網供電池，充電轉換前 AC 功率
$bH_{i,t}$	batteryToHomeKW	電池供自家負載，放電轉換後 AC 功率
$bS_{i,t}$	batteryToGridKW	電池分享輸出，放電轉換後、分享損失前
$E_{i,t}$	socKWh	時段開始時電池電量，單位 kWh
$yC_{i,t}$ 與 $yD_{i,t}$	chargeOn / dischargeOn	充電與放電旗標，各為 0 或 1

八個功率變數都不能是負的。其中七個對應示意圖上的電力路徑， u 是另外列出的棄光量。分享出去的電在程式裡叫 Grid，但在這個模型裡是拿來抵掉社區其他住戶的買電，沒有賣電的收入。

四個效率 η_{PA} 、 η_{PD} 、 η_C 、 η_D 都設為 0.95，分別對應 PV 到 AC、PV 到電池 DC、電網充電和電池放電。分享損失 $\xi = 0.07$ ，所以分享後剩下 $1 - \xi = 0.93$ 。效率只在電實際走過的路徑上套用，不能把每個 0.95 都乘到所有的電量上。

程式來源：src/solve_storenet.m 第 44–65 行；src/storenet_config.m 的主模型參數。

共同限制之一 電力平衡與棄光

PV 可以發的電要分到自用、充電、分享和棄光四個地方。前三項的變數記的是轉換後的量，所以要除以對應的效率，才會換回同一個 DC 側的基準。下面的式子對每一戶、每一個時段都成立。

$$\frac{pH_{i,t}}{\eta_{PA}} + \frac{pS_{i,t}}{\eta_{PA}} + \frac{pB_{i,t}}{\eta_{PD}} + u_{i,t} = A_{i,t}$$
$$gH_{i,t} + pH_{i,t} + bH_{i,t} = L_{i,t}$$

社區總買電 G_t 是各戶向電網買電的總和，再扣掉分享後能交給其他住戶的電。這是整個社區加總的能量帳，沒有另外模擬每一條配電線路。

$$G_t = \sum_{i=1}^N [gH_{i,t} + gB_{i,t} - (1 - \xi)(pS_{i,t} + bS_{i,t})]$$

$$G_t \geq 0, u_{i,t} \geq 0$$

```
aggregateImportKW = sum(gridToHomeKW + gridToBatteryKW - ...
    (1 - config.transferLossFraction) .* (pvToGridKW + batteryToGridKW), 2);

problem.Constraints.pvAllocation = ...
    pvToHomeKW ./ config.etaPvAC + pvToGridKW ./ config.etaPvAC + ...
    pvToBatteryKW ./ config.etaPvDC + pvCurtailKW == pvKW;
problem.Constraints.homeBalance = ...
    gridToHomeKW + pvToHomeKW + batteryToHomeKW == loadKW;
problem.Constraints.aggregateImportNonnegative = aggregateImportKW >= 0;
```

$G_t \geq 0$ 代表社區整體不會反過來向電網送電。 $u \geq 0$ 讓多出來的 PV 可以放棄不用，避免負載、電池和分享都吃不下時，模型還被迫把它全部用完。有沒有棄光要看排程；它和轉換損失、分享損失是不同的量。模型也沒有另外把「棄光最少」當成目標。

程式來源：src/solve_storenet.m 第 70–82、98 行；對照論文式 11、13、14。

共同限制之二 電池電量與充放電

先把充電和放電都換成電池端的功率。C 是進到電池的功率，D 是電池實際放出的功率；pB 已經是進到電池的量，所以不再乘一次充電效率。

$$C_{i,t} = \eta_C g B_{i,t} + p B_{i,t}, D_{i,t} = \frac{b H_{i,t} + b S_{i,t}}{\eta_D}$$

$$E_{i,t+1} = E_{i,t} + \Delta t (C_{i,t} - D_{i,t} - s_i)$$

$$1 \leq E_{i,t} \leq 9, E_{i,1} = E_{i,T+1} = 1 \text{ kWh}$$

$$0 \leq C_{i,t} \leq 3.3 y C_{i,t}, \quad 0 \leq D_{i,t} \leq 3.3 y D_{i,t}$$

$$y C_{i,t} + y D_{i,t} \leq 1, y C_{i,t}, y D_{i,t} \in \{0,1\}$$

```
chargePowerKW = config.etaBatteryCharge .* gridToBatteryKW + pvToBatteryKW;
dischargePowerKW = (batteryToHomeKW + batteryToGridKW) ./ ...
    config.etaBatteryDischarge;
problem.Constraints.socInitial = socKW(1, :) == ...
    config.socInitialFraction * config.batteryCapacityKW;
explicitSelfDischargeKW = getOptionalField(config, "selfDischargeKW", 0);
problem.Constraints.socDynamics = socKW(2:end, :) == socKW(1:end-1, :) + ...
    dtHours .* (chargePowerKW - dischargePowerKW - explicitSelfDischargeKW);
terminalFraction = getOptionalField(config, "socTerminalFraction", ...
    config.socInitialFraction);
problem.Constraints.socTerminal = socKW(end, :) == ...
    terminalFraction * config.batteryCapacityKW;
problem.Constraints.chargePowerLimit = chargePowerKW <= ...
    config.batteryPowerKW .* chargeOn;
problem.Constraints.dischargePowerLimit = dischargePowerKW <= ...
    config.batteryPowerKW .* dischargeOn;
problem.Constraints.chargeDischargeExclusion = chargeOn + dischargeOn <= 1;
```

每戶容量 10 kWh，SoC 限制在 10–90%，一天開始和結束都要回到 10%。s_i 是自放電功率，主模型設為 0。兩個旗標禁止同時充電又放電，但可以兩個都是零。頭尾電量相同，也避免把一開始電池裡的電當成免費的能源用掉。

程式來源：src/solve_storenet.m 第 67–69、83–97 行；主設定見 storenet_config.m。

共同目標與分步驟求解的誤差範圍

B 是這一天的度電費，Q 是所有電池的充電量加放電量。Q 是用來挑出比較少做多餘充放電的行程，不是換算成金額的電池老化成本。

$$B = \Delta t \sum_{t=1}^T \pi_t G_t, \quad Q = \Delta t \sum_{t=1}^T \sum_{i=1}^N (C_{i,t} + D_{i,t})$$

```
billExpression = dtHours .* sum(pricePerKWh .* aggregateImportKW);  
throughputExpression = dtHours .* sum(chargePowerKW + dischargePowerKW, "all");
```

電價 π_t 在時段的開始時間落在 $10:00 \leq \text{時間} < 22:00$ 時是 0.194 EUR/kWh，其他時候是 0.091。資料的 time 欄位是區間的結束時間，所以程式要先往前減一格，再判斷日夜電價和 PSDT 的日間範圍。

```
intervalStartTime = intervalEndTime - hours(dtHours);  
mask = hour(intervalStartTime) >= config.dayStartHour & ...  
       hour(intervalStartTime) < config.dayEndHour;
```

如果前一步的目標 f 算出最佳值 f^* ，下一步就加上下面這條限制。 ε 會隨目標值的大小縮放；本文說的「電費不變」，就是指在這個誤差範圍內不變。

$$f(x) \leq f^* + \varepsilon(f^*), \quad \varepsilon(v) = 10^{-7} \max(1, |v|)$$

```
function allowance = lexicographicAllowance(value, config)  
allowance = config.lexicographicTolerance .* max(1, abs(double(value)));  
end  
  
problem.Objective = objective;  
[rawSolution, fval, exitFlag, output] = solve(problem, "Options", options);
```

程式用 `intlinprog` 求解， $\text{RelativeGapTolerance} = 10^{-6}$ 、 $\text{ConstraintTolerance} = 10^{-8}$ 。每一步都會檢查求解狀態並記下 gap，最後再由另一支獨立的程式重算電費、尖峰和限制式的殘差。這些設定是我們自己定的，原論文沒有提供求解器的細節。

程式來源：src/solve_storenet.m 的 tariffForTime、daytimeMask、solveStage、lexicographicAllowance；src/evaluate_storenet.m。

SH BM 各戶自己的電費最低

SH-BM 把每一戶當成獨立的使用者，只能用自家的 PV 和電池抵掉自己的買電。先禁止兩條分享路徑，再把每戶的買電費加總求最小；因為住戶之間沒有任何關聯，這和每戶各自求最小是一樣的。

$$pS_{i,t} = bS_{i,t} = 0$$

$$B_i = \Delta t \sum_{t=1}^T \pi_t (gH_{i,t} + gB_{i,t}), \quad \min \sum_{i=1}^N B_i$$

求解順序 $B \rightarrow Q$

第二階段 $\min Q, \quad B \leq B^* + \varepsilon(B^*)$

```
problem.Constraints.noPvSharing = pvToGridKW == 0;
problem.Constraints.noBatterySharing = batteryToGridKW == 0;

[stageOneSolution, billValue, exitFlag, output] = solveStage(problem, ...
    billExpression, options, "bill");
problem.Constraints.billLexicographic = billExpression <= ...
    billValue + lexicographicAllowance(billValue, config);
throughputWeight = getOptionalField(config, "throughputWeight", 1);
[rawSolution, weightedThroughput, exitFlag, output] = solveStage(problem, ...
    throughputWeight .* throughputExpression, options, ...
    "throughput after bill");
```

這裡明確補上論文式 16 疑似漏掉的一項。把負載平衡代進去， $gH = L - pH - bH$ ，所以買電費會扣掉 PV 自用的部分。如果照刊出的式子漏掉 pH ，自家 PV 對電費的貢獻就不完整；我們也補上 Δt ，把功率換成計費用的能量。

程式來源：src/solve_storenet.m 的 SH_BM 限制與帳單分支；對照論文式 16。本頁的 $B \rightarrow Q$ 是我們的取捨規則。

VPP BM 整個社區的電費最低

VPP-BM 允許 PV 和電池在社區內分享。算式還是電價乘社區買電功率，再照時間加總，只是 G_t 已經扣掉分享後有效的那部分電。分享讓調度多了選擇，但主目標本身沒有要求尖峰要低。

$$\min B = \min \Delta t \sum_{t=1}^T \pi_t G_t$$

求解順序 $B \rightarrow Q$

第二階段 $\min Q, \quad B \leq B^* + \varepsilon(B^*)$

```
[stageOneSolution, billValue, exitFlag, output] = solveStage(problem, ...  
    billExpression, options, "bill");  
problem.Constraints.billLexicographic = billExpression <= ...  
    billValue + lexicographicAllowance(billValue, config);  
throughputWeight = getOptionalField(config, "throughputWeight", 1);  
[rawSolution, weightedThroughput, exitFlag, output] = solveStage(problem, ...  
    throughputWeight .* throughputExpression, options, ...  
    "throughput after bill");
```

這一段和 SH-BM 的電費求解完全一樣；差別只在進到這段程式之前，有沒有加上「不能分享」的兩條限制。第一步求出最低電費後，第二步只減少電池的充放電量。它不會去比較兩個電費相同的排程誰的最高功率比較小。

這也說明了第二版要補的地方：最低電費可能對應很多種充電時間的安排。只要都在便宜的時段充電，電費幾乎一樣，充電集中的程度卻可以差很多。

程式來源：src/solve_storenet.m 的帳單分支；對照論文式 17。

PS 全天最高進口功率最低

引入 z 當全天買電功率的上限。每個時段都不能超過 z ，再把 z 壓到最低。主目標完成後，我們的程式才比電費，最後減少充放電量。

$$\min z, \quad 0 \leq G_t \leq z \quad \forall t, \quad \text{求解順序} \quad z \rightarrow B \rightarrow Q$$

$$B^* = \min B, \quad z \leq z^* + \varepsilon(z^*)$$

$$\min Q, \quad B \leq B^* + \varepsilon(B^*)$$

```
systemPeakKW = optimvar("systemPeakKW", 1, "LowerBound", 0);
problem.Constraints.systemPeak = aggregateImportKW <= systemPeakKW;
[stageOneSolution, primaryValue, exitFlag, output] = solveStage(problem, ...
    systemPeakKW, options, "system peak");
problem.Constraints.systemPeakLexicographic = systemPeakKW <= ...
    primaryValue + lexicographicAllowance(primaryValue, config);
[stageTwoSolution, billValue, exitFlag, output] = solveStage(problem, ...
    billExpression, options, "bill after system peak");
problem.Constraints.billLexicographic = billExpression <= ...
    billValue + lexicographicAllowance(billValue, config);
throughputWeight = getOptionalField(config, "throughputWeight", 1);
[rawSolution, weightedThroughput, exitFlag, output] = solveStage(problem, ...
    throughputWeight .* throughputExpression, options, ...
    "throughput after system peak and bill");
```

PS 願意多付電費來降低全天的最高點。它和第二版的差別在順序：PS 先保住最低尖峰，第二版先保住最低電費。

程式來源：src/solve_storenet.m 的 PS 分支；對照論文式 18–19。

PSDT 只壓低日間最高進口功率

日間集合 $\mathcal{D}$ 包含開始時間落在 10:00 到 22:00 之間的時段。只對這些時段設尖峰上限 z_D ；夜間不受這條限制，但還是要遵守共同的模型。

$$\min z_D, G_t \leq z_D \forall t \in \mathcal{D}, \quad \text{求解順序 } z_D \rightarrow B \rightarrow Q$$

$$B^* = \min B, z_D \leq z_D^* + \varepsilon(z_D^*)$$

$$\min Q, \quad B \leq B^* + \varepsilon(B^*)$$

```
daytimePeakKW = optimvar("daytimePeakKW", 1, "LowerBound", 0);
problem.Constraints.daytimePeak = ...
    aggregateImportKW(dayMask) <= daytimePeakKW;
[stageOneSolution, primaryValue, exitFlag, output] = solveStage(problem, ...
    daytimePeakKW, options, "daytime peak");
problem.Constraints.daytimePeakLexicographic = daytimePeakKW <= ...
    primaryValue + lexicographicAllowance(primaryValue, config);
[stageTwoSolution, billValue, exitFlag, output] = solveStage(problem, ...
    billExpression, options, "bill after daytime peak");
problem.Constraints.billLexicographic = billExpression <= ...
    billValue + lexicographicAllowance(billValue, config);
throughputWeight = getOptionalField(config, "throughputWeight", 1);
[rawSolution, weightedThroughput, exitFlag, output] = solveStage(problem, ...
    throughputWeight .* throughputExpression, options, ...
    "throughput after daytime peak and bill");
```

白天的尖峰變低，不代表整天的最高點也會降。電池可能把充電集中到晚上；看 PSDT 的圖時，白天和晚上的曲線要一起看。

程式來源：src/solve_storenet.m 的 PSDT 分支及 daytimeMask；對照論文式 20–22。

LL 把全天最高與最低的差縮小

LL 同時引入最高值的上限 z 和最低值的下限 m ，讓所有的 G_t 都落在中間，再把 $z-m$ 縮到最小。這個目標看的是整條曲線的高低差，和只壓最高點的 PS 不一樣。

$$\min (z - m), \quad 0 \leq m \leq G_t \leq z \quad \forall t$$

$$\text{求解順序} \quad (z - m) \rightarrow B \rightarrow Q$$

$$B^* = \min B, \quad z - m \leq (z - m)^* + \varepsilon((z - m)^*)$$

$$\min Q, \quad B \leq B^* + \varepsilon(B^*)$$

```
maximumImportKW = optimvar("maximumImportKW", 1, "LowerBound", 0);
minimumImportKW = optimvar("minimumImportKW", 1, "LowerBound", 0);
importSpreadKW = maximumImportKW - minimumImportKW;
problem.Constraints.maximumImport = ...
    aggregateImportKW <= maximumImportKW;
problem.Constraints.minimumImport = ...
    aggregateImportKW >= minimumImportKW;
[stageOneSolution, primaryValue, exitFlag, output] = solveStage(problem, ...
    importSpreadKW, options, "import spread");
problem.Constraints.importSpreadLexicographic = importSpreadKW <= ...
    primaryValue + lexicographicAllowance(primaryValue, config);
[stageTwoSolution, billValue, exitFlag, output] = solveStage(problem, ...
    billExpression, options, "bill after import spread");
problem.Constraints.billLexicographic = billExpression <= ...
    billValue + lexicographicAllowance(billValue, config);
throughputWeight = getOptionalField(config, "throughputWeight", 1);
[rawSolution, weightedThroughput, exitFlag, output] = solveStage(problem, ...
    throughputWeight .* throughputExpression, options, ...
    "throughput after import spread and bill");
```

把最高點降低，或把原本很低的谷底墊高，都可以縮小高低差。所以曲線比較平，不代表用電量或電費一定最少。

程式來源：src/solve_storenet.m 的 LL 分支；對照論文式 23–24。

Peak Guard 在最低電費問題加上上限

先用同一天的負載和 PV，算出「各戶 PV 自用、沒有電池、沒有分享」時的社區買電。這條基準線的最高點 P_0 就是上限；它不是還沒扣掉 PV 的原始負載尖峰。

$$G_t^0 = \sum_{i=1}^N \max(L_{i,t} - \eta_{PA} A_{i,t}, 0), P_0 = \max_t G_t^0$$
$$\min B, G_t \leq P_0 \forall t, \quad \text{求解順序} \quad B \rightarrow Q$$

$$\text{第二階段} \quad \min Q, \quad B \leq B^* + \varepsilon(B^*)$$

```
% Same-day PV-self/no-battery baseline
pvACAvailableKW = config.etaPvAC .* data.pvKW;
pvSelfToHomeKW = min(data.loadKW, pvACAvailableKW);
baselineImportKW = sum(data.loadKW - pvSelfToHomeKW, 2);
config.aggregateImportCapKW = max(baselineImportKW);

% Constraint inside solve_storenet
importCapKW = validateImportCap(config, intervalCount);
problem.Constraints.aggregateImportCap = ...
    aggregateImportKW <= importCapKW;

[stageOneSolution, billValue, exitFlag, output] = solveStage(problem, ...
    billExpression, options, "bill");
problem.Constraints.billLexicographic = billExpression <= ...
    billValue + lexicographicAllowance(billValue, config);
throughputWeight = getOptionalField(config, "throughputWeight", 1);
[rawSolution, weightedThroughput, exitFlag, output] = solveStage(problem, ...
    throughputWeight .* throughputExpression, options, ...
    "throughput after bill");
```

上限先把可以用的排程範圍縮小，再在這個範圍內找最低電費。如果原本電費最低的那些排程都守不住 P_0 ，Peak Guard 就得多付錢。代表日的上限約 17.94 kW，多付的電費約 1.0131 EUR。

程式來源：上面的基準是照 `evaluate_storenet.m` 整理的等價寫法；上限與求解來自 `solve_storenet.m`。

第二版 先固定最低電費再削峰

第二版 VPP-BM+peak 不先指定 P_0 。它先解和 VPP-BM 完全相同的電費問題，再在最低電費的誤差範圍內找最低尖峰，最後才減少充放電量。用 x 代表整組排程，三個步驟是：

$$B^* = \min B(x)$$

$$z^* = \min z, \quad B(x) \leq B^* + \varepsilon(B^*), G_t \leq z$$

$$\min Q(x), \quad B(x) \leq B^* + \varepsilon(B^*), \quad z \leq z^* + \varepsilon(z^*)$$

```
[stageOneSolution, billValue, exitFlag, output] = solveStage(problem, ...
    billExpression, options, "bill");
problem.Constraints.billLexicographic = billExpression <= ...
    billValue + lexicographicAllowance(billValue, config);
systemPeakKW = optimvar("systemPeakKW", 1, "LowerBound", 0);
problem.Constraints.systemPeak = aggregateImportKW <= systemPeakKW;
[stageTwoSolution, peakValue, exitFlag, output] = solveStage(problem, ...
    systemPeakKW, options, "system peak after bill");
problem.Constraints.systemPeakLexicographic = systemPeakKW <= ...
    peakValue + lexicographicAllowance(peakValue, config);
throughputWeight = getOptionalField(config, "throughputWeight", 1);
[rawSolution, weightedThroughput, exitFlag, output] = solveStage(problem, ...
    throughputWeight .* throughputExpression, options, ...
    "throughput after bill and system peak");
```

尖峰變數是在第一步之後才加進來的，所以第一步和原本的 VPP-BM 完全一樣。這樣能找出不增加電費的削峰空間，但不保證最後的尖峰一定低於 P_0 。

程式來源：src/solve_storenet.m 的 VPP_BM_PEAK_LEX 分支；設計紀錄 ADR-002。

上限掃描 把更多削峰的代價算出來

第二版找到同電費的最低尖峰後，再一步一步收緊上限，看度電費要增加多少。r 是我們事先指定的比例；每一個 r 都是一個獨立的 Peak Guard 求解問題。

$$B_r^* = \min B(x), G_t \leq r P_0 \forall t$$

$$\Delta B_r = B_r^* - B_{VPP}^*$$

```
% P0 has already been calculated from the same-day baseline.
ratios = [3, 2.5, 2, 1.75, 1.5, 1.25, 1.1, 1, ...
          0.95, 0.9, 0.85, 0.8, 0.75, 0.7, 0.65, 0.6, 0.55, 0.5];
for r = ratios
    caseConfig = config;
    caseConfig.aggregateImportCapKW = r * P0;
    [solution, metrics] = solve_storennet(data, caseConfig, ...
        "IMPROVED_PEAK_GUARD");
    % Save metrics.optimizedBillEUR and metrics.peakImportKW.
end
```

上面是方便閱讀的呼叫範例。實際執行的程式會逐案保存結果，分成成功、資料被排除、無解和求解失敗四種，不會因為某個上限無解就偷偷放寬。實際程式設定上限的部分如下：

```
if planned.Strategy == "IMPROVED_PEAK_GUARD"
    config.aggregateImportCapKW = ...
        planned.CapRatio .* baseline.pvSelfPeakKW;
end
```

r = 1 就是第一版的 Peak Guard。r 越小要求越嚴，但不保證每個 r 都有解。有解的時候，第二步還是固定電費後減少充放電量，不會再把尖峰壓到上限以下的最低值；所以某個不用付錢的上限點，不能直接當成 VPP-BM+peak。

ΔB_r 是用同一天、同一批資料和參數的 VPP-BM 當基準，兩者的電費相減。比較很多天時，也要配對相同的有解日期，不能把少數幾天的總代價直接和完整 45 天比。

程式來源：src/run_bahloul_frontier_v1.m 的 CapRatios、planCases、configureCase；上限問題由 solve_storennet.m 求解。

需量費模型 用價格表達尖峰的代價

這個延伸不直接規定最高功率，改成假設每天照尖峰收費。 λ 是每天每 kW 的需量費率， λz 是當天的需量費；把它和原本的度電費 B 相加，聚合商就會同時考慮這兩筆錢。

$$\min J = B + \lambda z, G_t \leq z \quad \forall t, \quad \lambda \geq 0$$

求解順序 $J \rightarrow Q$

$$\text{第二階段} \quad \min Q, \quad B + \lambda z \leq J^* + \varepsilon(J^*)$$

```
demandChargeEURPerKW = validateDemandCharge(config);
systemPeakKW = optimvar("systemPeakKW", 1, "LowerBound", 0);
problem.Constraints.systemPeak = aggregateImportKW <= systemPeakKW;
weightedExpression = billExpression + ...
    demandChargeEURPerKW .* systemPeakKW;
[stageOneSolution, weightedValue, exitFlag, output] = solveStage(problem, ...
    weightedExpression, options, "bill plus demand charge");
problem.Constraints.demandChargeLexicographic = weightedExpression <= ...
    weightedValue + lexicographicAllowance(weightedValue, config);
throughputWeight = getOptionalField(config, "throughputWeight", 1);
[rawSolution, weightedThroughput, exitFlag, output] = solveStage(problem, ...
    throughputWeight .* throughputExpression, options, ...
    "throughput after bill plus demand charge");
```

λ 的單位是 EUR / kW / 日，決策的範圍是一天。這不是照整個月最高需量收費的模型。要模擬月需量費，得讓整個月的時段共用同一個尖峰變數，不能把每天的最佳解直接加起來當成月的模型。

報告裡的「增加度電費」只比較 B ，沒有把 λz 加進那一欄；求解目標則已經包含 λz 。比較經濟代價時，要分清楚電池調度多出來的買電費，和電價制度另外收的需量費。

程式來源：src/solve_storenet.m 的 VPP_BM_DEMAND_CHARGE 分支；費率網格見 run_bahloul_frontier_v1.m。

完整程式的入口與結果核對

所有求解都從 `solve_storenet(data, config, strategy)` 進入。`data.loadKW` 和 `data.pvKW` 是 $T \times N$ 的 kW 矩陣；`data.time` 是 $T \times 1$ 的區間結束時間，`data.dtHours` 是每格的小時數，`data.houseIds` 存 N 戶的名稱。下面是資料前處理完成後的呼叫方式。

```
strategies = ["SH_BM", "VPP_BM", "PS", "PSDT", "LL", ...  
             "VPP_BM_PEAK_LEX"];  
for strategy = strategies  
    [solution, metrics] = solve_storenet(data, config, strategy);  
    disp([metrics.optimizedBillEUR, metrics.peakImportKW]);  
end  
  
guardConfig = config;  
guardConfig.aggregateImportCapKW = P0;  
[guard, guardMetrics] = solve_storenet(data, guardConfig, ...  
                                       "IMPROVED_PEAK_GUARD");  
  
tariffConfig = config;  
tariffConfig.demandChargeEURPerKW = 1;  
[tariff, tariffMetrics] = solve_storenet(data, tariffConfig, ...  
                                       "VPP_BM_DEMAND_CHARGE");
```

表 23 完整程式的主要檔案

檔案	用途
src/storenet_config.m	效率、容量、電價與誤差範圍設定
src/load_storenet_day.m src/prepare_bahloul_case.m	公開資料的日期讀取、彙整與案例設定
src/solve_storenet.m	共同 MILP 與八個策略分支
src/evaluate_storenet.m	用輸入和解獨立重算結果及限制式誤差
src/run_bahloul_frontier_v1.m	第二版上限、費率與效率案例的逐案執行

核對算式和程式時，先確認 PV、電池和分享的量測位置，再核對 Δt 和日夜時間，最後才看每個目標的先後順序。結果檔另外存了每一步的目標值和限制式殘差，可以檢查前一步的結果有沒有被保住。較低效率的實驗只改 ηC 、 ηD ，不改五個原策略的定義。

附的程式包是專案 `src` 目錄目前的 MATLAB 原始碼，包含驗證和輸出結果的部分；官方原始資料和已經定案的結果留在原專案裡。這一版新增的是說明文字和程式對照，圖表沿用前面已經完成的實驗結果。

程式來源：完整 MATLAB 程式包 `StoreNet_策略程式.zip`；檔案雜湊與使用說明見包內 `SOURCE_MANIFEST.json`、`README.md`。

參考資料與結果來源

- [1] Trivedi, R., Bahloul, M., Saif, A., Patra, S., and Khadem, S. (2024). Comprehensive Dataset on Electrical Load Profiles for Energy Community in Ireland. *Scientific Data*, 11, 621. <https://doi.org/10.1038/s41597-024-03454-2>
- [2] Bahloul 等人 (2022). Role of Aggregator in Coordinating Residential Virtual Power Plant in StoreNet: A Pilot Project Case Study. *IEEE Transactions on Sustainable Energy*, 13(4), 2148–2158.
<https://doi.org/10.1109/TSTE.2022.3187217> 原圖擷取自專案保存的論文 PDF；模型見第 5 至 6 頁，Figure 5 至 7 與 Table I 見第 7 至 9 頁，附錄參數見第 10 頁。
- [3] Khadem, S., Trivedi, R., Bahloul, M., Saif, A., and Patra, S. (2024). Comprehensive Dataset on Electrical Load Profiles for Energy Community in Ireland. Figshare 官方資料集合，版本 1，共 46 項。<https://doi.org/10.6084/m9.figshare.c.6829134.v1> 截圖日期為 2026 年 9 月 5 日。
- [4] Saif, A., Khadem, S. K., Conlon, M., and Norton, B. (2023). Local Electricity Market operation in presence of residential energy storage in low voltage distribution network: Role of retail market pricing. *Energy Reports*, 9, 5799–5811. <https://doi.org/10.1016/j.egyr.2023.05.005> 資料論文 Figure 10 對應該文 Figure 13。
- [5] 資料論文圖表結果：StoreNet/results/data_paper_figures_5_10_v2/。圖表及對應的 CSV 同時保留原程式的算法與修正後的算法。說明文件：StoreNet/docs/DATA_PAPER_FIGURES_5_10.md。
- [6] 方法論文復現結果：StoreNet/results/b2022_ir_v1_formal/。包含代表日、81 格敏感度、Table I 及月度結果；R1 至 R8 驗收全部通過。版本標記 bahloul-vpp-formal-v1。
- [7] Peak Guard 與澳洲比較：StoreNet/results/bahloul_vpp_improvement_v1/，包含 storenet_typical、storenet_monthly、ausgrid_no161。版本標記 bahloul-vpp-peak-guard-v1。
- [8] 第二版改進：StoreNet/results/bahloul_vpp_improvement_v2_frontier/。代表日 39 案；月度 816 案都有狀態紀錄。版本標記 bahloul-vpp-frontier-v2。
- [9] Ratnam, E. L., Weller, S. R., Kellett, C. M., and Murray, A. T. Residential load and rooftop PV generation: an Australian distribution network dataset. <https://doi.org/10.1080/14786451.2015.1100196> 原發布：Ausgrid。
封存提供：Pierre Haessig（2026 年 5 月）；<https://pierrehaessig.eu/ausgrid-solar-data/>
原官方下載頁已失效；資料轉換與雜湊見 StoreNet/docs/AUSGRID_DATASET_CARD.md。